# Settings Reference

Every setting of every job: its default, what it does, and where to set it

Site Automation - read it next to the setup guide | site-automation 0.15.1 | October 2026

**What this is.** Every setting the site-automation jobs read - the value it has when you set nothing, and what it does - with how to use one: where to put it (a template's Variables, `playbooks/group_vars/all.yml`, a group or a host), which place wins, and how to see what a job used.

> **How to read it**
> Keep it open next to the *Operations Runbooks Setup Guide*: each job's chapter there says which settings matter for it; this one lists them all, role by role.
> Rendered from `docs/VARIABLES.md` and `docs/VARIABLES_REFERENCE.md` (generated from `roles/*/defaults/main.yml`) in the site-automation repository (version 0.15.1).

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

# Contents

1. Settings: what they are, where they go, and who wins 4

The two-minute version . . . 4
How to use a setting . . . 4
1. What you can set . . . 5
2. Where do I put a setting? . . . 7
3. Who wins when the same setting is in two places? . . . 8
Examples . . . 10
How to verify a setting is in effect . . . 10
If a setting does nothing . . . 11

2. Every setting and its default 12

ad_report . . . 12
check_accounts . . . 14
check_auditd . . . 14
check_certs . . . 15
check_containers . . . 19
check_disk . . . 20
check_fapolicyd . . . 21
check_logging . . . 21
check_mariadb . . . 22
check_mounts . . . 25
check_network . . . 25
check_patching . . . 26
check_performance . . . 26
check_selinux . . . 27
check_services . . . 28
check_time . . . 28
ontap_report . . . 28
patch . . . 30
poam . . . 31
podman_discover . . . 32
service_watch . . . 33
servicenow . . . 35

site_act . . . 37
site_email . . . 38
site_findings . . . 39
stigman_stack . . . 40
troubleshoot . . . 42
vmware_vm . . . 43
win_act . . . 58
win_check_accounts . . . 58
win_check_audit . . . 59
win_check_certs . . . 59
win_check_disk . . . 60
win_check_eventlog . . . 61
win_check_network . . . 61
win_check_patching . . . 62
win_check_performance . . . 62
win_check_security . . . 62
win_check_services . . . 63
win_check_time . . . 64
win_patch . . . 64
win_troubleshoot . . . 65

# 1. Settings: what they are, where they go, and who wins

A **variable** (also called a *setting*) is a named value the runbooks read: `check_disk_warn_pct: 85` means "warn when a disk is 85 % full". You change a variable to change how a check behaves, without touching any code. This page answers three questions:

1. **What can I set?** (find the name and the default)
2. **Where do I put it?** (for every host, for one group, for one host, for one run)
3. **What happens when the same setting is in two places?** (who wins)

You do not need to read it all. Do the two-minute version first.

## The two-minute version

1. **Find the setting.** Open VARIABLES_REFERENCE.md, press Ctrl+F and search for a word (`disk`, `certificate`, `mariadb`). Every setting is there with its default.
2. **Put the line in one file.**
   - Every host: `playbooks/group_vars/all.yml`
   - Hosts in one AAP group: `playbooks/group_vars/<group name>.yml`
   - One host: `playbooks/host_vars/<host name>.yml`
3. **Commit and Sync Changes** in VS Code. Run the check.

That is all for a normal change. The rest of this page explains the details and the exceptions.

**For one job template:** `aap/extra_vars/<playbook>.yml` (the list) has every setting that template reads, switched off, with its default and what it does. Copy the file into the template's **Variables** box (YAML selected), delete the `#` in front of the settings you change, and change their values. Switch on only what you change: a template's Variables beat `all.yml`.

## How to use a setting

Every job's chapter in these docs ends its settings part with an **All settings** link: the list of that job's settings in VARIABLES_REFERENCE.md, each with its default and what it does. Pick the place by **who the setting is for**:

| The setting is about | Put it in | Example |
|---|---|---|
| hosts (thresholds, what a check looks at) | `playbooks/group_vars/all.yml` (every host), `playbooks/group_vars/<group>.yml`, `playbooks/host_vars/<host>.yml` | `check_disk_warn_pct: 90` |
| one job template (who gets its email, its rules) | the job template's **Variables** box in AAP | `report_email_to: ops@yoursite.mil` on the certificate report template |
| one run | a **survey** question (its variable name = the setting's name) or **Prompt on launch** | a survey question `health_checks`, multiple choice `daily` / `weekly` |

Three examples:

1. playbooks/group_vars/all.yml - every host: disks warn at 90 %, a stricter audit log partition

```
check_disk_warn_pct: 90
check_disk_overrides:
  /var/log/audit: {warn: 70, crit: 85}

# 2. the "Certificate report" template's Variables - this job only: email it, stay green
report_email_to: [pki-team@yoursite.mil]
site_fail_on: []

# 3. playbooks/group_vars/win_web.yml - the Windows web servers only: IIS must run too
win_check_services_required_extra: [W3SVC, WAS]
```

- **A list replaces the default list** (it does not add to it): repeat the entries you keep. Details: "Three shapes of value" below.
- **Check what a job used:** in AAP, the job's **Details** tab shows the template's and the launch's variables; the job's output prints the decisions it made (thresholds in the finding texts, `REFUSED:` lines, the settings each report names in its email); its **artifacts** (Details > Artifacts) carry the results. A setting that seems ignored: If a setting does nothing.

## 1. What you can set

Every setting lives, with its default and an explanation, in the role that uses it: `roles/<role>/defaults/main.yml`. Two easy ways to read them:

- **VARIABLES_REFERENCE.md**, the same information as one searchable page (generated from the role files, so it is never out of date). Start here.
- In VS Code: **Ctrl+Shift+F** (search in all files), type the setting or a word, and look at the hits under `roles\...\defaults\main.yml`.

Setting names start with the thing they belong to:

| Starts with | Belongs to | Example |
|---|---|---|
| `check_disk_`, `check_services_`, `check_mariadb_` ... | one health check (`roles/check_<name>`) | `check_disk_warn_pct` |
| `win_check_` | one Windows health check | `win_check_disk_warn_pct` |
| `site_act_` | ACT (GenAI) | `site_act_level` |
| `site_` (other) | reporting and settings | `site_fail_on` |
| `patch_`, `win_patch_`, `automatic_restarts` | patching | `patch_never_patch_groups` |
| `servicenow_` | ServiceNow tickets | `servicenow_assignment_group` |
| `watch_` | service watch | `watch_containers`, `watch_discover` |
| `podman_discover_` | finding the podman containers (the `containers` check and service watch share it) | `podman_discover_ignore` |
| `poam_` | POA&M | `poam_stigman_api` |
| `stigman_` | STIG Manager deployment | `stigman_scope` |

Check names and what each finding means: RUNBOOKS.md.

### Three shapes of value

```yaml
check_disk_warn_pct: 85                    # a number
check_fapolicyd_required: true             # true or false (lower case)
check_services_required: [sshd, crond]     # a list: [a, b] or one "- item" per line
check_disk_overrides:                      # a set of names with values (a "map")
  /var/log/audit: {warn: 70, crit: 85}
```

**A list replaces the default list. It does not add to it.** If the default is `[sshd, crond, rsyslog]` and you write `[chronyd]`, the hosts now require **only** `chronyd`. Copy the default list from the reference and add your entries to it. (Some settings have a separate `..._extra` version that *adds*, for example `check_services_required_extra`. The reference says which.)

### Job variables (per run, not per host)

These choose what a run does. Put them in a job template's **Variables** box (*Extra variables*) or in a survey question, never in a settings file:

| Variable | Meaning | Where it appears |
|---|---|---|
| `target` | which hosts or group to run on (default: all) | most playbooks |
| `health_checks` | which checks: `daily`, `weekly`, `all`, or a list | Health check |
| `ts_area`, `ts_service`, ... | what to troubleshoot | Troubleshoot |
| `use_act`, `site_act_level` | ask ACT, and what it may do | Health check, Troubleshoot |
| `site_fail_on` | which findings fail the job: `[critical]` or `[critical, warning]` | any check job |

## 2. Where do I put a setting?

| Place | Who gets it | How you change it | Good for |
|---|---|---|---|
| `roles/<role>/defaults/main.yml` | everyone | do **not** edit (updates replace it) | the built-in default |
| `playbooks/group_vars/all.yml` | every host | VS Code, commit, sync | site-wide values: the LDAP server, the SIEM, thresholds |
| `playbooks/group_vars/<group>.yml` | the hosts of that AAP group | VS Code | one kind of server: STIG Manager containers, MariaDB, AAP itself |
| `playbooks/host_vars/<host>.yml` | that one host | VS Code (create the folder and file) | one odd server: a bigger disk limit for `/var/log/audit` |
| **AAP: inventory > Variables** | every host | AAP web UI | values you prefer to keep in AAP |
| **AAP: group > Variables** | the hosts of that group | AAP web UI | same, per group |
| **AAP: host > Variables** | that host | AAP web UI | same, per host |
| **Job template > Variables** (*Extra variables*) | every run of that template | AAP web UI | "this template always uses `site_act_level: diagnose`" |
| **Survey question / prompt on launch** | this launch only | asked at launch | a choice made each time |
| **AAP credential** | the job | AAP web UI | **secrets only** (SECRETS.md) |

**File names matter.** `playbooks/group_vars/<group>.yml` applies to the group with **exactly that name** in your AAP inventory, and `playbooks/host_vars/<host>.yml` to the host with exactly that name. A file for a group your inventory does not have is silently never used.

The files shipped in `playbooks/group_vars/` and the group each expects:

| File | Group name it applies to | If your group has another name |
|---|---|---|
| `all.yml` | every host | (always works) |
| `aap.yml` | `aap` | copy it to `aap_hosts.yml` if your group is `aap_hosts` (the example inventory's name) |
| `mariadb.yml` | `mariadb` | the MariaDB check also runs for a group named `mariadb_hosts`, but the **settings** come from the file named after the group: copy it to `mariadb_hosts.yml` |
| `sat.yml`, `idm.yml`, `logstash.yml`, `stigman.yml`, `rhel8_all.yml` | the same name | copy or rename |
| `netapp.yml` | `netapp` | copy to `netapp_console_hosts.yml` if that is your group |

(The built-in safety lists cover both spellings of the AAP and NetApp group names, so AAP stays protected whatever you call it.)

**Updating the repository never overwrites your settings files.** The update script (`scripts/update-from-release.ps1` / `.sh`) protects `playbooks/group_vars/`, `playbooks/host_vars/`, `inventories/site/`, `poam/poam.csv` and anything in `.site-local`. If a new release adds a settings file you do not have, the script adds it **once** (the preview lists it as `YOURS`) and never changes it again. A setting that the release's `all.yml` mentions and yours does not is listed under `NEW SETTINGS`, so you can copy it over if you want it.

### Adding "extras" to a host or a group in AAP

Where the boxes are in AAP 2.7 (older versions word a label differently):

- **Inventory**: **Automation Execution › Infrastructure › Inventories** › your inventory › **Edit inventory** › **Variables**.
- **Group**: the same inventory › **Groups** tab › the group › **Edit group** › **Variables**.
- **Host**: the same inventory › **Hosts** tab › the host › **Edit host** › **Variables**.
- **Job template**: **Automation Execution › Templates** › the template › **Edit template** › **Extra variables** (the template's *Variables* box).

The **Variables** box on a host or group takes the same YAML as a settings file. Typical entries:

a group's Variables box: hosts in this group need a bigger disk margin

```
check_disk_overrides:
  /var/lib/pgsql: {warn: 70, crit: 85}
```

a host's Variables box: its MariaDB containers belong to a rootless account

```
check_mariadb_container_user: svc_podman
```

a group's Variables box: never check or watch the test containers, or anything of one user

```
podman_discover_ignore: ['test-.*', 'devuser/.*']
```

Connection settings (`ansible_host`, `ansible_user`, `ansible_python_interpreter`, `ansible_connection: winrm`) belong here too. The *login and password* never do: they are credentials (SECRETS.md).

**Prefer the project's settings files** (this repository), because they are reviewed and versioned in Git and updates never overwrite them. Use the AAP boxes when AAP is where your team already manages the inventory. Either is fine; the rule is **one setting, one place**.

## 3. Who wins when the same setting is in two places?

Strongest first. This order was **proven by running a test playbook** on ansible-core 2.16 and 2.21 with the same value set in every place and removing the winner one place at a time (the same result on both versions).

| # | Place | Notes |
|---|---|---|
| 1 | **Extra variables**: job template *Variables*, a survey answer, a prompt on launch, `-e` on the command line | beats everything below, including the built-in protection further down |
| 2 | `playbooks/host_vars/<host>.yml` | this project, one host |

| # | Place | Notes |
|---|---|---|
| 3 | the **host's** Variables box in AAP | |
| 4 | `playbooks/group_vars/<group>.yml` | a child group beats its parent group |
| 5 | `playbooks/group_vars/all.yml` | |
| 6 | the **group's** Variables box in AAP | a deeper group beats its parent |
| 7 | the **inventory's** Variables box in AAP | |
| 8 | `roles/<role>/defaults/main.yml` | the built-in default |

Things that surprise people:

- **A file in this project beats the AAP group and inventory boxes**, even when the box is on a more specific group. If you type a value in an AAP box and see no effect, look for the same setting in `playbooks/group_vars/`. Every job prints a notice, `NOTE - settings in this project's files win over the same setting in AAP's inventory Variables box`, listing which files define which settings. (Silence it with `site_settings_notice: false`.) Not shown for `playbooks/host_vars/`.
- **A host's own Variables box (3) beats a group file (4/5)**, but a `playbooks/host_vars` file (2) beats the host's box.
- **A survey answer or template extra variable beats everything**: it is the one place that can override the rest for a single run.
- Two sibling groups (neither is inside the other) that set the same variable for the same host: do not rely on the result. Put it in a host file, or in one group.
- An inventory **sourced from the project** (`inventories/site/`, SETUP_AAP.md step 5): AAP imports its `group_vars/all.yml` into the inventory's Variables box and each `group_vars/<group>.yml` into that group's box. So they rank as places 7 and 6, and the project's `playbooks/` files beat them. Keep a setting in one of the two, not both.
- **Updated from 0.4.1 or older?** Your `playbooks/group_vars/all.yml` is yours, so the update left it as it was: it still sets `site_act_diagnose_only_groups`, `patch_never_patch_groups`, `patch_never_reboot_groups`, `site_act_provider` and `site_act_models` as real (uncommented) lines. They beat the same settings in AAP's inventory and group boxes. If you want those set in AAP (or the new built-in defaults), put `#` in front of those lines. See the upgrade notes in `CHANGELOG.md` (0.5.0).

### Built-in protection: settings that cannot be weakened from a file

Some lists are protected in code, so a mistake in a settings file cannot expose the AAP host:

| What | Built in | What your file can do |
|---|---|---|
| Never patched, never rebooted | `aap`, `aap_hosts` (`roles/patch/vars/main.yml`) | your `patch_never_patch_groups` / `patch_never_reboot_groups` list **replaces** the shipped one (`sat`, `netapp`, ...): repeat the names you keep. `aap` and `aap_hosts` stay protected whatever you write |
| ACT never fixes by itself | `aap`, `aap_hosts` (`roles/site_act/vars/main.yml`) | your `site_act_diagnose_only_groups` list **replaces** the shipped one (`sat`, `idm`, `netapp`, ...): repeat the names you keep. `aap` and `aap_hosts` stay protected whatever you write |

Only an extra variable (place 1) can override these, and nothing in these docs asks you to do that.

## Examples

**Everyone: warn earlier on disks.** `playbooks/group_vars/all.yml`:

```yaml
check_disk_warn_pct: 80
check_disk_crit_pct: 92
```

**One group: the STIG Manager hosts need these containers watched.**
`playbooks/group_vars/stigman.yml` (shipped; already has `watch_containers`).

**One host: a database server whose audit partition must stay emptier.**
`playbooks/host_vars/db01.example.mil.yml` (the file name is the host's name in AAP):

```yaml
check_disk_overrides:
  /var/log/audit: {warn: 70, crit: 85}
```

**One template: the "Troubleshoot" template always uses Ask Sage.** In the template's **Variables**:

```yaml
site_act_provider: asksage
```

**A list you want to extend.** Where the role has a `..._extra` setting, use it: it *adds*.

```yaml
check_services_required_extra: [tomcat]        # required on top of the built-in five
```

Where it has none, look up the default (`check_services_required` is `[sshd, crond, rsyslog, chronyd, auditd]`) and write the whole list with your addition:

```yaml
check_services_required: [sshd, crond, rsyslog, chronyd, auditd, tomcat]
```

## How to verify a setting is in effect

1. **Run the check on one host**: **Health check** with `target` = that host and `health_checks` = the check. The finding text quotes the limit it used (`/var is 96% full (critical at 92%)`), so you see the value that won.
2. **Break it on purpose** (or lower the limit) on a test host, and confirm the finding appears. A check that never fires proves nothing.
3. **Read the job's notice** (`NOTE - settings in this project's files ...`): it names every settings file that set something, so you can spot a file that overrides an AAP box.
4. **From the command line** (optional, in a terminal with Ansible and the repository): create a temporary file `playbooks/show_settings.yml`, **do not commit it**, and run it:

YAML

```yaml
- name: Show the settings a host gets
  hosts: "{{ target | default('all') }}"
  gather_facts: false
  tasks:
    - ansible.builtin.import_role: {name: check_disk}    # the role that owns the setting
      when: false                                        # loads its defaults, runs nothing
    - ansible.builtin.debug:
        msg: "{{ item }} = {{ lookup('vars', item, default='(not set)') }}"
      loop: [check_disk_warn_pct, check_disk_crit_pct]
```

shell

```
ansible-playbook -i inventories/site/hosts.yml playbooks/show_settings.yml -e
  target=db01.example.mil
```

It shows the value from the role default plus the files in this project. (Values typed into AAP's Variables boxes exist only inside AAP; there, use steps 1-3.)

## If a setting does nothing

| Symptom | Likely cause |
|---|---|
| You set it in AAP and nothing changes | the same setting is in `playbooks/group_vars/`: that file wins (table above) |
| You set it in a file and nothing changes | the file name does not match the group or host name exactly; a spelling or capital letter |
| A list lost entries | a list replaces the default: repeat the entries you keep |
| YAML error at the start of the job | indentation (spaces, never tabs), a missing space after `:`, or a value with `:` inside it that needs quotes |
| A survey answer is ignored | the survey question's **variable name** is not exactly the setting's name |

# 2. Every setting and its default

This page lists **every setting a role reads**, with the value it has when you set nothing, and what the role's own comment says. It is generated from `roles/<role>/defaults/main.yml`, so it is never out of date. Where to put a setting, and which place wins when it is in two, is in VARIABLES.md. Values are shortened to one line here: the role file has the full one.

A setting that is a list (`[a, b]`) **replaces** the default list when you set it; it does not add to it. Repeat the entries you want to keep.

**Contents:** ad_report, check_accounts, check_auditd, check_certs, check_containers, check_disk, check_fapolicyd, check_logging, check_mariadb, check_mounts, check_network, check_patching, check_performance, check_selinux, check_services, check_time, ontap_report, patch, poam, podman_discover, service_watch, servicenow, site_act, site_email, site_findings, stigman_stack, troubleshoot, vmware_vm, win_act, win_check_accounts, win_check_audit, win_check_certs, win_check_disk, win_check_eventlog, win_check_network, win_check_patching, win_check_performance, win_check_security, win_check_services, win_check_time, win_patch, win_troubleshoot

## ad_report

File: `roles/ad_report/defaults/main.yml`

> **Note**
> Active Directory health report (playbooks/ad_health_report.yml). Read-only: a PowerShell script on every domain controller (WinRM, the Windows Machine credential) and one more on one DC per domain. The account needs to read AD and run dcdiag / repadmin on the DCs: see docs/ACTIVE_DIRECTORY.md.

| Setting | Default | What it does |
|---|---|---|
| `ad_report_hours` | `24` | ---- Changes ---------- the changes of the last this-many hours (match it to the schedule: a daily job = 24) |
| `ad_event_hours` | `24` | the event log entries of the last this-many hours |
| `ad_security_events` | `true` | read each DC's Security log for WHO made a change (needs auditing of account and security group management) |
| `ad_max_events` | `2000` | read at most this many entries per event log and DC |
| `ad_changes_max` | `500` | look at most at this many changed objects per kind (created, changed accounts, GPO links) |
| `ad_privileged_groups_extra` | `[DnsAdmins ]` | Privileged groups watched besides the built-in ones (Domain / Enterprise / Schema Admins, Administrators, Account / Server / Print / Backup Operators, Group Policy Creator Owners, Key Admins, Cert Publishers): names or SIDs of this domain's groups. |
| `ad_groups_should_be_empty` | `[]` | privileged groups that must have no member, e.g. [Schema Admins] - amber when they have one |

| Setting | Default | What it does |
|---|---|---|
| ad_privileged_change_severity | warning | a member added to / removed from a privileged group: warning, critical or info |
| ad_schema_change_severity | warning | the schema version changed: warning, critical or info |
| ad_dc_change_severity | info | a domain controller added or removed: warning, critical or info |
| ad_dcdiag_skip | [SystemLog] | ---- What is read on each DC ---------- dcdiag tests to skip (SystemLog fails on any error in the System log: the Events table covers it) |
| ad_dcdiag_dns | true | also dcdiag's DNS tests that only read (basic, forwarders, delegation, record registration) |
| ad_trust_verify | true | verify outbound and two-way trusts (nltest /sc_verify) |
| ad_repl_warn_hours | 6 | ---- Thresholds: at or above "warn" = amber, at or above "crit" = red ---------- a replication link with no success for this long (inter-site schedules: keep above their interval) |
| ad_repl_crit_hours | 24 | no success for this long (or a failing link) -> red |
| ad_repl_queue_warn | 50 | replication requests waiting in a DC's inbound queue |
| ad_time_warn_seconds | 5 | a DC's clock against the PDC emulator (the PDC: against its NTP source) |
| ad_time_crit_seconds | 60 | clock offset -> red (Kerberos fails at 300) |
| ad_time_sync_warn_hours | 24 | Windows Time last synchronized longer ago than this |
| ad_disk_warn_pct | 15 | free space of the database, log and SYSVOL volumes, % (at or below) |
| ad_disk_crit_pct | 5 | free % -> red |
| ad_cert_warn_days | 30 | the LDAPS / KDC certificate expires within this many days |
| ad_cert_crit_days | 7 | expires within this many days -> red |
| ad_backup_warn_days | 7 | the last system-state backup of a partition is older than this |
| ad_backup_crit_days | 30 | older than this -> red (always red past the tombstone lifetime) |
| ad_krbtgt_max_days | 180 | the krbtgt password is older than this -> amber (the STIG asks for 180) |
| ad_rid_warn_pct | 10 | RID pool left, % (at or below) |
| ad_rid_crit_pct | 2 | RID pool left -> red |
| ad_no_client_site_warn | 1 | this many client addresses in no AD subnet -> amber (fewer: blue) |

| Setting | Default | What it does |
|---|---|---|
| ad_report_max_rows | 50 | rows per table (problems first); the job's artifacts have all |
| ad_report_fail | false | The job stays green when it finds problems: the report and email say what. true = mark it FAILED when there is a critical finding or a DC could not be read, for workflows. |

## check_accounts

File: roles/check_accounts/defaults/main.yml

> **Note**
> ACCOUNTS: the account problems auditors and attackers both look for, plus the one that quietly breaks automation - a service account password about to expire.

| Setting | Default | What it does |
|---|---|---|
| check_accounts_min_uid | 1000 | people's accounts start here |
| check_accounts_inactive_days | 35 | the STIG wants unused accounts disabled after 35 days |
| check_accounts_ignore_inactive | [] | Accounts that may stay unused (break-glass, service accounts that never log in interactively) |
| check_accounts_nopasswd_allowed | [] | sudoers entries allowed to use NOPASSWD, e.g. the account AAP connects as: [svc_aap, "%wheel"] |
| check_accounts_expiry_watch | [] | Accounts whose password/account expiry matters - above all the account AAP logs in with. e.g. [svc_aap] |
| check_accounts_expiry_warn_days | 14 | a watched account's password or account expiring within this many days -> warning (3 days or less -> critical) |

## check_auditd

File: roles/check_auditd/defaults/main.yml

> **Note**
> AUDITD: the audit system records who did what (the STIG depends on it). Running, enabled in the kernel, rules loaded, no events lost, room for its log, and (optionally) forwarding. Also how MUCH it records: too much rotates the evidence away within hours, fills the disk (then auditd stops the host or puts it in single-user mode, as the STIG sets it up), drops records on the way to the SIEM, and uses up the SIEM licence.

| Setting | Default | What it does |
|---|---|---|
| check_auditd_min_rules | 5 | fewer loaded rules than this = the STIG rules are missing |
| check_auditd_log_warn_pct | 75 | the audit log's filesystem |
| check_auditd_log_crit_pct | 90 | % used of that filesystem -> critical (warning from check_auditd_log_warn_pct) |
| check_auditd_remote_required | false | true = au-remote (audisp-remote) must be active |
| check_auditd_exclude_accounts | [] | Accounts whose audit events are expected in bulk and do NOT count toward the volume warning, e.g. the account a vulnerability scanner logs in with (a credentialed scan runs thousands of commands with sudo). Names or UIDs; a host that does not have the account skips it. The report still says how much they made, and the... (all of it: the role file) |
| check_auditd_volume_hours | 24 | look at the audit records of the last N hours |
| check_auditd_volume_warn_mb_day | 200 | warning above this many MB of audit records a day (excluded accounts not counted) |
| check_auditd_retention_warn_hours | 24 | With max_log_file_action ROTATE, auditd keeps num_logs files of max_log_file MB and deletes the oldest. Warning when what is left on disk reaches back fewer hours than this. The STIG wants a week (168) when the records are NOT sent off the host as they are written (RHEL-08-030660). 0 = off. |
| check_auditd_volume_timeout | 120 | The breakdown (which account, which rule, which program) reads the audit log files of the window at low CPU and disk priority, and only when the volume or the retention is over its limit. It stops after this many seconds on a host with very large logs (and says so). |

## check_certs

File: roles/check_certs/defaults/main.yml

> **Note**
> CERTS: certificates that expire soon - as files on the host, inside Java keystores, and as served by TLS endpoints (what a browser or client actually sees).

| Setting | Default | What it does |
|---|---|---|
| check_certs_warn_days | 30 | a certificate expiring within this many days -> warning (and amber in the certificate report) |
| check_certs_notice_days | 60 | the certificate report colours these blue (an early heads-up; not a finding) |
| check_certs_crit_days | 14 | within this many days (or expired) -> critical |
| check_certs_files | [] | Certificate files (PEM) to check. Globs work. Every certificate in a file is checked (a chain file with an expiring intermediate is a problem too). e.g. [/etc/stigman/*.crt, /etc/pki/tls/certs/server.crt] |
| check_certs_discover | true | Also look for certificates in the usual places. Files with many certificates are CA bundles (hundreds of roots, noisy) and are skipped unless listed in check_certs_files. |
| check_certs_discover_dirs | [/etc/pki/tls/certs, /etc/pki/nginx, /etc/nginx, /etc/httpd/conf.d, /etc/stigman, /etc/st… | the folders searched (two levels deep) for *.crt, *.pem, *.cer (/etc/stigman/nginx/tls: STIG Manager's nginx certificate, three levels down) |
| check_certs_discover_exclude | 'ca-bundle\|ca-certificates\|/localhost\.crt$' | regular expression: found files whose path matches are skipped |
| check_certs_discover_max_per_file | 10 | a found file with more certificates than this is a CA bundle: skipped |
| check_certs_placeholder_subject | '(^\|,)O=SomeOrganization(,\|$)' | Regular expression on the subject of a package's placeholder certificate: postfix's /etc/pki/tls/certs/postfix.pem (O=SomeOrganization), self-signed for a year at install and never renewed. A found one is listed as not checked (no finding, no ticket) and none gets a CSR. A real certificate put in its place is checked;... (all of it: the role file) |
| check_certs_endpoints | [] | TLS endpoints, checked FROM this host, e.g. the site's own web server: check_certs_endpoints: - {name: STIG Manager, host: 127.0.0.1, port: 443, servername: stigman.example.mil} |

| Setting | Default | What it does |
|---|---|---|
| check_certs_keystores | [] | Java keystores (Tomcat, the ServiceNow MID Server ...). A JKS keystore lists without a password; a PKCS12 one needs it - give the NAME of an environment variable that holds it (set by an AAP credential), never the password itself: check_certs_keystores: - {path: /opt/servicenow/mid/agent/security/agent_keystore, passw... (all of it: the role file) |
| check_certs_discover_ports | false | ---- Finding them by themselves (the certificate report turns both on; the health check does not) ---- Listening TCP ports that answer TLS: each probed once, on the address it listens on, with `openssl s_client` (check_certs_probe_timeout seconds each, at most check_certs_discover_ports_max ports). Not these ports: pl... (all of it: the role file) |
| check_certs_discover_ports_skip | [22, 25, 53, 80, 110, 111, 123, 143, 323, 389, 631, 2049, 3306, 5355, 5432, 6379, 11211,… | ports never probed (plain-text services: ssh, smtp, dns, http, ldap, databases ...) |
| check_certs_discover_ports_max | 25 | probe at most this many listening ports (the rest are listed as not probed) |
| check_certs_probe_timeout | 3 | seconds for each port probe |
| check_certs_discover_keystores | false | Java keystores (*.jks, *.keystore, *.p12, *.pfx) in these folders, and the ones running Java programs use (-Djavax.net.ssl.keyStore=...). A keystore that needs a password is listed as not checked: no password is ever guessed - give it in check_certs_keystores (password_env). The JDK's own trust stores (cacerts) are sk... (all of it: the role file) |
| check_certs_discover_keystore_dirs | [/etc, /opt, /srv, /usr/share/tomcat, /var/lib/tomcat, /var/lib/pki] | the folders searched for keystores |
| check_certs_discover_keystore_depth | 5 | how many folder levels deep the search goes |
| check_certs_discover_keystore_exclude | '/(cacerts\|jssecacerts)$\|/jre/lib/security/\|/lib/security/' | regular expression: keystore paths skipped (the JDK's trust stores) |

| Setting | Default | What it does |
|---|---|---|
| check_certs_discover_keystore_max | 40 | read at most this many found keystores per host |
| cert_skip_hosts | [] | ---- Leaving hosts out (the certificate report and the renewal; not the health check) ---- Hosts whose certificates something else renews - Satellite and its Capsules (satellite-installer, never by hand), appliances - are left out of the certificate report and never get a CSR: listed as not checked, no ticket. Name th... (all of it: the role file) |
| cert_skip_groups | [sat] | AAP inventory groups, e.g. [sat] (the health check still watches the Satellite web UI: group_vars/sat.yml) |
| cert_renew_dir | /etc/pki/tls/renew | ---- Renewal: a new key and CSR for each certificate file that expires soon (playbooks/cert_renew_csr.yml) ---- The certificates the report flags (within check_certs_warn_days - 30 days or less - or expired) get a new key and a CSR with the same subject and names (subjectAltName), on the host; the CSR is attached to t... (all of it: the role file) |
| cert_renew_key_bits | 2048 | RSA key size, at least 2048 (a bigger old key keeps its size; an EC certificate gets a new key on the same curve) |
| cert_renew_digest | sha256 | the CSR's signature: sha256, sha384 or sha512 |
| cert_renew_max | 10 | at most this many NEW CSRs per host in one run (the rest come the next run) |
| cert_renew_only | (empty) | regular expression: only certificate files whose path matches (empty = every file), e.g. '^/etc/pki/tls/certs/' |
| cert_renew_issuer | (empty) | regular expression: only certificates whose issuer matches (empty = any CA), e.g. 'O=U\.S\. Government' |
| cert_renew_exclude | '/(katello\|foreman\|foreman-proxy\|candlepin\|pulp\|puppet\|puppetlabs\|rhsm\|qpid)/\|/etc/pki/(c… | Certificates something else renews - never a CSR from here. The path (Satellite, Capsule, Puppet, subscription-manager on every registered host) and the issuer (Satellite's own CA, Puppet's CA). Check the report's Where column for your hosts' paths and add yours. |

| Setting | Default | What it does |
|---|---|---|
| cert_renew_exclude_issuer | 'O=Katello\|Puppet CA' | regular expression on the issuer: Satellite's own CA (O=Katello), Puppet's CA - never a CSR from here |
| cert_renew_self_signed | false | Self-signed certificates (made on the host by an application or a person, not by a PKI) are reported and get a ticket, but no CSR - unless this is true: then the PKI is asked for a real certificate in their place (a STIG wants PKI certificates). Never a CA's own certificate (a root: it signs certificates), never one o… (all of it: the role file) |
| cert_renew_install_notes | /etc/stigman/: >- STIG Manager: put the issued certificate (with its chain) and the new k… | What to do with the issued certificate when something else deploys the file (a copy by hand would be overwritten): the start of the file's path -> the next step written on the ticket instead of "install it in place of the file". Add your own. |
| cert_renew_attach_ext | csr | The CSR's attachment on the ticket: <common name>-<8 characters of the CSR's SHA-256>.<ext> (a new CSR, a new name). An instance that refuses .csr attachments (its allowed extensions): txt. Refused, the CSR goes in a work note. |
| cert_renew_attach_type | text/plain | the attachment's Content-Type (a CSR is Base64 text) |

## check_containers

File: roles/check_containers/defaults/main.yml

> **Note**
> CONTAINERS: the podman containers on the host - root's (rootful) and every user's (rootless) - found by themselves (roles/podman_discover: which ones exist, which systemd unit runs each one, which should be running). Read-only. Which containers and users are looked at, and which are left out: the podman_discover_* settings.

| Setting | Default | What it does |
|---|---|---|
| check_containers_required | [] | Containers that MUST exist and run on these hosts: a container name, or OWNER/NAME for one owner (root/web, alice/web). A name alone matches that container of any owner. Also matches a Quadlet file name without .container, e.g. [stigman-mysql, stigman-api, alice/web]. |

| Setting | Default | What it does |
|---|---|---|
| check_containers_recent_hours | 24 | A container that stopped with an error (exit code not 0) this many hours ago or less is reported (containers:crashed). Older crashes are history and are not reported. |
| check_containers_restart_warn | 3 | More restarts than this (podman's restart count, or its systemd unit's) = a restart loop. |

## check_disk

File: roles/check_disk/defaults/main.yml

> **Note**
> DISK: how full each LOCAL filesystem is (space and inodes), and - from the history of earlier runs - how many days until it is full. Network filesystems (NFS) are the mounts check's job: a hung NFS server can make df hang, so this check never touches them.

| Setting | Default | What it does |
|---|---|---|
| check_disk_warn_pct | 85 | % used -> warning |
| check_disk_crit_pct | 95 | % used -> critical |
| check_disk_inode_warn_pct | 85 | inodes: too many small files fill a disk too |
| check_disk_inode_crit_pct | 95 | % of inodes used -> critical (warning from check_disk_inode_warn_pct) |
| check_disk_overrides | {} | Different limits for particular mount points, e.g. keep the audit log partition emptier: check_disk_overrides: /var/log/audit: {warn: 70, crit: 85} |
| check_disk_ignore | - '^/(proc\|sys\|dev\|run)(/\|$)' - '^/etc/(hosts\|hostname\|resolv\.conf)$' # bind-mounted fil… | Mount points to skip (regular expressions). |
| check_disk_exclude_types | [tmpfs, devtmpfs, squashfs, overlay, iso9660, efivarfs, nsfs] | filesystem types never checked (df -x) |
| check_disk_forecast | true | Days-until-full forecast. Each run adds a sample (date, bytes used) per mount to a small file on the host; once there is at least a day of history, the growth rate predicts when the filesystem fills. Check mode (a dry run) reads the history but does not add to it. |

| Setting | Default | What it does |
|---|---|---|
| check_disk_forecast_warn_days | 14 | full within this many days at the current growth -> warning |
| check_disk_forecast_crit_days | 3 | full within this many days -> critical |
| check_disk_history_file | /var/lib/site-health/disk-history.json | where the forecast keeps its samples, on the host |
| check_disk_history_keep_days | 30 | samples older than this are dropped (the forecast uses what is left) |
| check_disk_timeout | 20 | seconds for df |

## check_fapolicyd

File: roles/check_fapolicyd/defaults/main.yml

> **Note**
> FAPOLICYD (application allow-listing): running, enforcing, configuration and trust database valid, and recent denials - which is what you see when it blocks a program.

| Setting | Default | What it does |
|---|---|---|
| check_fapolicyd_required | false | true = every host in the play must have fapolicyd. false = only check hosts that have it. |
| check_fapolicyd_expect_enforcing | true | permissive = 1 means it only logs, never blocks |
| check_fapolicyd_check_trustdb | true | Compare the trust database with the files on disk (can take a minute on big hosts; files changed outside RPM show up here, e.g. after a manual update). |
| check_fapolicyd_denials_since | yesterday | ausearch -ts value |
| check_fapolicyd_denials_warn | 1 | this many denials or more since check_fapolicyd_denials_since -> warning |

## check_logging

File: roles/check_logging/defaults/main.yml

> **Note**
> LOGGING: are logs kept, and do they reach the central log server (SIEM)?

| Setting | Default | What it does |
|---|---|---|
| check_logging_journal_persistent | true | the journal must survive a reboot (/var/log/journal) |

| Setting | Default | What it does |
|---|---|---|
| check_logging_remote | [] | The central log server(s) this host forwards to, e.g. check_logging_remote: - {name: SIEM, host: siem.example.mil, port: 6514} |
| check_logging_timeout | 5 | seconds to wait for each log server's TCP port |
| check_logging_queue_warn_mb | 100 | rsyslog keeps messages it cannot deliver in /var/lib/rsyslog. A big queue = the log server has been unreachable for a while. |
| check_logging_errors_since | "-24h" | Error-priority journal messages in the last day. Many = something is complaining a lot. |
| check_logging_errors_warn | 500 | this many error messages or more -> warning |

## check_mariadb

File: roles/check_mariadb/defaults/main.yml

> **Note**
> DATABASE health for MariaDB AND MySQL (read-only; role name kept for compatibility): running, accepting connections, connection headroom, queries that run too long, replication (MariaDB multi-source, MySQL REPLICA/SLAVE), Galera / Percona XtraDB Cluster, MySQL Group Replication, recent errors in its log, and room on its disk. It runs status queries only - no data is read and nothing is changed. The engine and version are detected; see docs/MARIADB.md.

| Setting | Default | What it does |
|---|---|---|
| check_mariadb_enabled | (worked out by the role; see its file) | Which hosts: members of any of the inventory groups mariadb_hosts, mysql_hosts, database_hosts, mariadb or mysql, or any host that names a container below (or turns discovery on). Set this true/false to override for a host or group. |
| check_mariadb_container | (empty) | MariaDB or MySQL in podman containers (e.g. the ServiceNow database). The client inside each container is used (podman exec). Name them in one of these ways; all are checked, each as its own instance: check_mariadb_container: mariadb one container check_mariadb_containers: [mariadb, snow-db] several containers on the... (all of it: the role file) |
| check_mariadb_containers | [] | several containers on one host, e.g. [mariadb, snow-db] (see above) |

| Setting | Default | What it does |
|---|---|---|
| check_mariadb_container_discover | auto | auto, true or false: find database containers by their image (see above) |
| check_mariadb_server_paths | [/usr/sbin/mariadbd, /usr/libexec/mariadbd, /usr/sbin/mysqld, /usr/libexec/mysqld, /usr/b… | Where a database server installed on the host lives (any one present = a host install, for auto). |
| check_mariadb_container_image_regex | '(mariadb\|mysql\|percona)' | regular expression: a container whose image matches is a database (discovery) |
| check_mariadb_container_user | (empty) | ROOTLESS podman: the containers belong to a user (e.g. AAP's own MariaDB, or an application account), and only that user can see them. Name the user and the check runs podman AS that user. Empty = root's containers. |
| check_mariadb_service | (empty) | The systemd unit (host installs). Empty = the first of mariadb, mysqld, mysql that exists. |
| check_mariadb_client_args | [] | How to log in. Without a credential: as root through the local socket - MariaDB's default for a host install on RHEL (no password needed when the job runs with sudo). A database in a CONTAINER usually gives root a password (the official images do), and MySQL host installs usually do too, so there attach the "MariaDB m… (all of it: the role file) |
| check_mariadb_conn_warn_pct | 80 | connections in use, % of max_connections |
| check_mariadb_conn_crit_pct | 95 | connections in use, % of max_connections -> critical |
| check_mariadb_threads_running_warn | 32 | queries executing at this moment |
| check_mariadb_long_query_seconds | 300 | a query running longer than this is "long" |
| check_mariadb_replica_lag_warn | 300 | seconds a replica is behind its primary |
| check_mariadb_replica_lag_crit | 1800 | seconds behind its primary -> critical |
| check_mariadb_galera_size | 0 | expected Galera / XtraDB Cluster size (0 = do not check the size) |
| check_mariadb_group_size | 0 | expected MySQL Group Replication members (0 = do not check the size) |
| check_mariadb_restart_warn_minutes | 60 | restarted less than this ago -> warning |

| Setting | Default | What it does |
|---|---|---|
| check_mariadb_bufferpool_hit_warn | 95 | % of reads served from memory (only judged after 1M reads) |
| check_mariadb_error_since_days | 1 | [ERROR] lines in its log this recent |
| check_mariadb_disk_warn_pct | 85 | the filesystem holding the data directory |
| check_mariadb_disk_crit_pct | 95 | % used of the data directory's filesystem -> critical |
| check_mariadb_performance | true | ---- PERFORMANCE (all warnings) ---------- ---------- false = only the health checks above. Counters that count since the server started (slow queries, temporary tables, joins, lock waits, deadlocks ...) are judged as an average since that start, and only once it has run chec... (all of it: the role file) |
| check_mariadb_perf_min_uptime_hours | 2 | judge the averages since the start only after this many hours |
| check_mariadb_slow_per_hour_warn | 60 | queries slower than long_query_time, per hour |
| check_mariadb_tmp_disk_pct_warn | 25 | % of temporary tables that went to disk (judged after 1000 of them) |
| check_mariadb_full_join_per_hour_warn | 100 | joins without a usable index (Select_full_join), per hour |
| check_mariadb_table_lock_wait_pct_warn | 1 | % of table locks (MyISAM / Aria) that had to wait (judged after 10000) |
| check_mariadb_row_lock_avg_ms_warn | 500 | average InnoDB row lock wait, ms (judged after 100 waits) |
| check_mariadb_deadlocks_per_hour_warn | 1 | InnoDB deadlocks per hour (judged after 10 of them) |
| check_mariadb_innodb_waits_per_hour_warn | 1 | waits for a free buffer pool page / a full log buffer, per hour (after 10) |
| check_mariadb_aborted_connects_per_hour_warn | 60 | failed connection attempts per hour |
| check_mariadb_table_open_per_hour_warn | 600 | tables opened per hour while the table cache is full |
| check_mariadb_open_files_warn_pct | 85 | files open, % of open_files_limit |
| check_mariadb_lock_wait_seconds | 30 | a session waiting this long for a lock, right now |
| check_mariadb_long_trx_seconds | 600 | an InnoDB transaction open this long, right now |

| Setting | Default | What it does |
|---|---|---|
| check_mariadb_history_length_warn | 1000000 | InnoDB history list length (old row versions not yet purged) |

## check_mounts

File: roles/check_mounts/defaults/main.yml

> **Note**
> MOUNTS: filesystems that should be mounted but are not, local filesystems that went read-only (the kernel does that after disk errors), and network mounts (NFS/CIFS) that hang. Diagnose only: this check never mounts, unmounts or remounts anything.

| Setting | Default | What it does |
|---|---|---|
| check_mounts_network_types | [nfs, nfs4, cifs, smb3] | filesystem types counted as network mounts (checked for hangs and fullness) |
| check_mounts_local_types | [xfs, ext4, ext3, btrfs, vfat] | filesystem types checked for having gone read-only |
| check_mounts_timeout | 5 | seconds before a network mount counts as hung |
| check_mounts_warn_pct | 85 | Network mounts: how full (same idea as the disk check) |
| check_mounts_crit_pct | 95 | % used of a network mount -> critical (warning from check_mounts_warn_pct) |
| check_mounts_ignore | [] | fstab mount points that are allowed to be unmounted (regular expressions) |

## check_network

File: roles/check_network/defaults/main.yml

> **Note**
> NETWORK: can this host reach what it depends on? A default route, names that must resolve, and TCP ports that must answer (from THIS host, which is what matters when a service breaks).

| Setting | Default | What it does |
|---|---|---|
| check_network_require_default_route | true | no default route -> finding (false for an isolated host) |
| check_network_resolve_self | true | this host's own full name (hostname -f) resolves |
| check_network_dns_names | [] | e.g. [ldap.example.mil, servicenow.example.mil] |

| Setting | Default | What it does |
|---|---|---|
| check_network_tcp | [] | TCP services this host needs, e.g. check_network_tcp: - {name: LDAP, host: ldap.example.mil, port: 636} - {name: SIEM (syslog over TLS), host: siem.example.mil, port: 6514} |
| check_network_timeout | 5 | seconds for each name lookup and TCP port test |

## check_patching

File: roles/check_patching/defaults/main.yml

> **Note**
> PATCHING (readiness, read-only): pending security updates, a reboot still needed after the last updates, and how long ago the host was last updated. Installs nothing - patching itself is playbooks/patch_hosts.yml.

| Setting | Default | What it does |
|---|---|---|
| check_patching_max_age_days | 35 | last package update older than this -> warning |
| check_patching_security_warn | 1 | this many pending security advisories -> warning |
| check_patching_critical_is_critical | true | a pending Critical-severity advisory -> critical |
| check_patching_timeout | 300 | seconds for dnf to talk to the repositories |
| check_patching_reboot_check | true | report a reboot still needed (needs-restarting -r, newer kernel installed than running) |

## check_performance

File: roles/check_performance/defaults/main.yml

> **Note**
> PERFORMANCE: is the host overloaded right now, or was something killed for lack of memory?

| Setting | Default | What it does |
|---|---|---|
| check_performance_load_warn | 1.5 | 15-minute load average PER CPU (1.0 = every CPU busy) |
| check_performance_load_crit | 3.0 | load per CPU -> critical |
| check_performance_mem_avail_warn_pct | 10 | memory still available for programs, % of total |

| Setting | Default | What it does |
|---|---|---|
| check_performance_mem_avail_crit_pct | 5 | available memory below this % -> critical |
| check_performance_swap_warn_pct | 50 | swap in use, % of swap |
| check_performance_swap_crit_pct | 80 | swap in use above this % -> critical |
| check_performance_iowait_warn | 20 | % of CPU time waiting for disk (average over the sample) |
| check_performance_iowait_crit | 40 | I/O wait above this % -> critical |
| check_performance_steal_warn | 10 | % of CPU taken by the hypervisor (VM starved) |
| check_performance_sample_seconds | 5 | how long vmstat samples |
| check_performance_oom_since | "-24h" | look for out-of-memory kills this far back |
| check_performance_zombie_warn | 20 | this many zombie processes or more -> warning |

## check_selinux

File: roles/check_selinux/defaults/main.yml

> **Note**
> SELINUX: enforcing (the STIG requires it), the config file agrees (so a reboot does not change it), no locally added permissive domains, and recent denials (AVCs) - which is what you see when SELinux blocks an application.

| Setting | Default | What it does |
|---|---|---|
| check_selinux_expected_mode | enforcing | the mode SELinux must be in (and the config file must say); anything else -> critical |
| check_selinux_avc_since | yesterday | ausearch -ts: recent (10 min), today, yesterday, this-week |
| check_selinux_avc_warn | 1 | this many denials or more -> warning |
| check_selinux_booleans | {} | Booleans that must have a value, e.g. {httpd_can_network_connect: "on"} |
| check_selinux_relabel_paths | [] | Paths whose file labels are compared with the policy (a dry-run restorecon). Mislabeled files are a common cause of denials after files are copied or moved. Keep it to small trees. e.g. [/etc/nginx, /var/www, /etc/stigman] |

## check_services

File: `roles/check_services/defaults/main.yml`

> **Note**
> SERVICES: systemd units that failed, services that must be running (and start at boot), and services stuck restarting over and over.

| Setting | Default | What it does |
|---|---|---|
| `check_services_required` | `[sshd, crond, rsyslog, chronyd, auditd]` | Must be running on every host. A host where one is not installed gets a warning (set check_services_required in the inventory to change the list for a group of hosts). |
| `check_services_required_extra` | `[]` | More for a group of hosts, e.g. in group_vars/mariadb_hosts.yml: [mariadb] |
| `check_services_ignore_failed` | `[]` | Failed units to ignore (regular expressions), e.g. a known-broken vendor unit. |
| `check_services_restart_warn` | `3` | More restarts than this since the unit started = a restart loop |

## check_time

File: `roles/check_time/defaults/main.yml`

> **Note**
> TIME: is the clock synchronized, and by how much is it off? A clock that is off by minutes breaks Kerberos/AD logins (5 minutes), TLS, log correlation and audit evidence.

| Setting | Default | What it does |
|---|---|---|
| `check_time_offset_warn` | `0.5` | seconds |
| `check_time_offset_crit` | `5.0` | seconds off -> critical (warning from check_time_offset_warn) |
| `check_time_min_sources` | `1` | time servers chrony must be able to reach |

## ontap_report

File: `roles/ontap_report/defaults/main.yml`

> **Note**
> NetApp ONTAP health report (playbooks/ontap_health_report.yml). Read-only: GET requests to each cluster's REST API, with the account of a "NetApp ONTAP" credential (docs/NETAPP.md).

| Setting | Default | What it does |
|---|---|---|
| ontap_clusters | [] | The clusters' management addresses (names or IPs), e.g. [cluster1.yoursite.mil, cluster2.yoursite.mil]. The credential's account must exist on each of them. |
| ontap_validate_certs | true | Check the clusters' HTTPS certificates. ONTAP's own certificate is often self-signed: then put its CA in ontap_ca_path (a file in the project, e.g. "{{ playbook_dir }}/files/ca/ontap-ca.pem"), or set false. |
| ontap_ca_path | (empty) | the CA file (PEM) the clusters' certificates are checked with; empty = the execution environment's own trust |
| ontap_timeout | 30 | seconds for one request |
| ontap_volume_warn_pct | 85 | Thresholds (used %, hours, days, seconds): at or above "warn" = amber, at or above "crit" = red. |
| ontap_volume_crit_pct | 90 | volume used % -> red |
| ontap_aggr_warn_pct | 85 | aggregate used % -> amber |
| ontap_aggr_crit_pct | 90 | aggregate used % -> red |
| ontap_inode_warn_pct | 85 | inodes used % of a volume -> amber |
| ontap_inode_crit_pct | 90 | inodes used % -> red |
| ontap_snapmirror_lag_warn_hours | 24 | last good transfer longer ago than this |
| ontap_snapmirror_lag_crit_hours | 48 | last good transfer longer ago than this -> red |
| ontap_snapshot_reserve_warn_pct | 100 | snapshots using this much of their reserve (100 = they spill into the volume) |
| ontap_snapshot_reserve_crit_pct | 150 | snapshots using this much of their reserve -> red |
| ontap_cert_warn_days | 60 | certificates expiring within this many days |
| ontap_cert_crit_days | 14 | certificates expiring within this many days (or expired) -> red |
| ontap_time_drift_warn_seconds | 60 | a node's clock against the AAP server's |
| ontap_time_drift_crit_seconds | 300 | a node's clock this far off -> red |
| ontap_port_errors_warn | 1 | receive errors (CRC and other bad frames) since the node booted |
| ontap_port_errors_crit | 10000 | receive errors on a port since boot -> red |
| ontap_min_spares | 1 | fewer spare disks than this on a node = amber (0 = red) |
| ontap_ems_hours | 24 | the error events of the last this-many hours |

| Setting | Default | What it does |
|---|---|---|
| ontap_ems_max | 500 | read at most this many events per cluster |
| ontap_report_max_rows | 50 | rows per table (problems first); the job's artifacts have all |
| ontap_snapshot_top | 25 | the snapshot space table: the volumes whose snapshots use the most |
| ontap_report_fail | false | The job stays green when it finds problems: the report and email say what. true = mark it FAILED when there is a critical finding or a cluster could not be read, for workflows. |

## patch

File: roles/patch/defaults/main.yml

> **Note**
> PATCH: install updates with dnf, reboot if the updates need it, and prove the host came back: every service that was running before must be running after. One host at a time by default; the rollout stops at the first host that fails.

| Setting | Default | What it does |
|---|---|---|
| patch_packages | ['*'] | what to update ('*' = everything that has an update) |
| patch_security_only | false | true = only updates that fix security advisories |
| patch_exclude | [] | never update these here, e.g. ['mariadb*', 'kernel*'] |
| automatic_restarts | false | NO AUTOMATIC RESTARTS unless you say so: this playbook reboots a host only when automatic_restarts is true (set it in the job template's Variables, or answer the survey), and then only as patch_reboot says. With automatic_restarts false (the default) it installs the updates and tells you which hosts still need a reboo... (all of it: the role file) |
| patch_reboot | when_needed | Reboot (only with automatic_restarts: true): when_needed (needs-restarting says so) \| never \| always |
| patch_reboot_timeout | 1800 | seconds to wait for the host to come back |

| Setting | Default | What it does |
|---|---|---|
| patch_never_reboot_groups | [aap, aap_hosts, sat, netapp, netapp_console_hosts] | Hosts in these groups are patched but NEVER rebooted by this playbook (reboot them by hand, in a maintenance window). Add your own group names; the built-in protection below is on always. |
| patch_never_patch_groups | [no_patch, aap, aap_hosts, sat, netapp, netapp_console_hosts] | Hosts in these groups are skipped entirely (Satellite is upgraded with satellite-maintain, not dnf). |
| patch_min_free_mb | 1024 | BUILT IN, whatever you set above (roles/patch/vars/main.yml): the AAP hosts (groups aap and aap_hosts) are never patched and never rebooted by a job, and neither is the machine the job runs from (a local connection). A package script can restart services under the running job. Set the two lists above only to ADD group... (all of it: the role file) |
| patch_service_wait | 120 | seconds to wait for services to come back after a reboot |
| patch_ignore_services | '^(user@\|user-runtime-dir@\|session-\|dnf-makecache\|packagekit\|fwupd\|run-\|systemd-(hostname… | Services that come and go on their own and are not compared before/after (regular expression) |

## poam

File: roles/poam/defaults/main.yml

| Setting | Default | What it does |
|---|---|---|
| poam_csv | "{{ playbook_dir }}/../poam/poam.csv" | POA&M STATUS: which Plan of Action & Milestones items are overdue or due soon, and (optional) which open CAT I / CAT II findings in STIG Manager have no POA&M item yet. The POA&M list is a CSV file in your work Git (export from eMASS or your spreadsheet, keep the columns below, save as CSV). POA&M data is sensitive: k... (all of it: the role file) |
| poam_columns | id: POAM ID # unique item number weakness: Weakness # what is wrong severity: Severity #… | Column names in YOUR file for each field this check needs (change the right-hand side). |

| Setting | Default | What it does |
|---|---|---|
| poam_closed_statuses | [Completed, Closed, Risk Accepted, Cancelled] | items with one of these statuses are done: never reported |
| poam_due_soon_days | 30 | an open item due within this many days -> warning (past due -> critical) |
| poam_date_formats | ['%Y-%m-%d', '%m/%d/%Y', '%m/%d/%y', '%d-%b-%Y', '%d %b %Y', '%b %d, %Y'] | the date formats your file may use (Python strptime), tried in this order |
| poam_stigman_api | (empty) | ---- STIG Manager cross-check (optional) ---------- Needs a Keycloak client that may use the client-credentials grant, with the scope stig-manager:collection:read, and a grant in the collections to read. Your realm owner creates it - this repository never changes the realm. T... (all of it: the role file) |
| poam_stigman_collections | [] | collection names; empty = every collection the client can read |
| poam_stigman_severities | [high, medium] | CAT I and CAT II findings must have a POA&M item |
| poam_stigman_scope | (empty) | Scope to ask for. Empty = the client's default scopes (they must include stig-manager:collection:read or stig-manager:collection). |
| poam_validate_certs | true | check STIG Manager's and Keycloak's HTTPS certificates |

## podman_discover

File: roles/podman_discover/defaults/main.yml

> **Note**
> PODMAN DISCOVERY (read-only): every podman container on the host, root's (rootful) and each user's (rootless), with how it is run (a systemd unit or plain podman) and whether it should be running. Used by the Health check containers and by Service watch. It changes nothing.

| Setting | Default | What it does |
|---|---|---|
| podman_discover_enabled | true | false = do not look for containers at all (the containers check then says "skipped", and Service watch watches only its watch_containers list). |

| Setting | Default | What it does |
|---|---|---|
| podman_discover_rootless | true | Also look at the containers of users (rootless podman). A user is looked at when a podman process (conmon) runs as them, when they have "linger" on (/var/lib/systemd/linger/NAME), or when they are in /etc/subuid AND have container storage (~/.local/share/containers/storage). Their containers are read AS that user (run... (all of it: the role file) |
| podman_discover_users | [] | More users to look at (names), e.g. an application account the rules above miss. |
| podman_discover_ignore | [] | Containers to leave out, as regular expressions that must match the WHOLE container name or OWNER/NAME, e.g. ['test-.*'] *(every container whose name starts with test-, any owner), ['alice/.*']* (all of alice's containers), ['root/web'] (root's web only). |
| podman_discover_quadlet_dirs | [/etc/containers/systemd, /usr/share/containers/systemd] | Where root's Quadlet files are (*.container, and *.kube: a pod from Kubernetes YAML). A user's are always read from ~/.config/containers/systemd, /etc/containers/systemd/users/UID and /etc/containers/systemd/users. |
| podman_discover_unit_dirs | [/etc/systemd/system, /usr/local/lib/systemd/system] | Where root's own systemd units are looked for: a *.service that runs podman (podman generate systemd, podman run, podman kube play, podman pod start, podman-compose / docker-compose), and the podman-kube@ units enabled there (.wants links). A user's: ~/.config/systemd/user. Packaged units (/usr/lib/systemd/sys... (all of it: the role file) |
| podman_discover_timeout | 60 | Seconds one podman or systemctl call may take before it is given up (reported as a finding). |

## service_watch

File: roles/service_watch/defaults/main.yml

| Setting | Default | What it does |
|---|---|---|
| watch_discover | true | ---- What to watch ---------- With watch_discover on (the default), service watch finds the containers itself: every podman container on the host that SHOULD be running - root's (rootful) and each user's (rootless) - as the Health check `containers` finds th... (all of it: the role file) |
| watch_containers | [] | Containers to watch in any case, in dependency order (a container's dependencies first). They are watched before the discovered ones, and one that is missing is reported as down. name: the container name, as `podman ps -a` shows it (or its Quadlet / service name) user: optional: the user that owns it (rootless podman)... (all of it: the role file) |
| watch_url_ok_status | [200, 301, 302, 401, 403] | HTTP codes that count as "answering" |
| watch_find_units | true | ---- How each container is run (found by itself) ---------- A container that a systemd unit runs (a Quadlet .container file, `podman generate systemd`, or a unit you wrote) must be started with systemctl, never with `podman start`: the unit owns it, and while the unit is stopped the co... (all of it: the role file) |
| watch_quadlet_dirs | [/etc/containers/systemd, /usr/share/containers/systemd] | where root's Quadlet files are looked for (to find the unit that runs a container) |
| watch_mode | approval | ---- What happens when one is down ---------- approval: ACT only diagnoses. Its proposed fix waits for the AAP approval step, then service_fix_approved.yml runs exactly the approved command(s) - no model involved. When ACT proposes nothing (or cannot run), the standard fix... (all of it: the role file) |
| watch_verify_retries | 6 | re-checks after a fix (every watch_verify_delay seconds) |
| watch_verify_delay | 5 | seconds between those re-checks |
| watch_record_file | /var/log/service-watch/incidents.jsonl | ---- Where incidents are recorded (on the host) ---------- One JSON line per event, plus a syslog line (tag service-watch) that your log forwarding (Splunk, Elastic, rsyslog relay) picks up. AAP's job history keeps its own copy. |

| Setting | Default | What it does |
|---|---|---|
| watch_record_syslog | true | also send each incident to syslog (tag service-watch) |
| watch_act_env | {} | ---- ACT settings ---------- Which provider and model: site_act_provider / site_act_model / site_act_url / site_act_ca, the same settings every runbook uses (roles/site_act/defaults/main.yml). The key comes from the "ACT model key" AAP credential, never f... (all of it: the role file) |
| watch_act_timeout | 1200 | seconds for one ACT run (the model's time per answer: site_act_model_timeout) |
| watch_instructions | (worked out by the role; see its file) | What ACT is told, on top of the act_triage role's standard instructions. |

## servicenow

File: roles/servicenow/defaults/main.yml

> **Note**
> SERVICENOW - open, update (and optionally resolve) incidents from check findings, and check that ServiceNow itself (the instance and its MID Servers) is healthy. Uses the REST Table API with ansible.builtin.uri: no extra collection to install. Connection: the "ServiceNow API" AAP credential sets SN_HOST (https://<instance>), SN_USERNAME and SN_PASSWORD in the job's environment. Without it, nothing is sent: the job prints what it WOULD have opened (a safe way to try it out).

| Setting | Default | What it does |
|---|---|---|
| servicenow_min_severity | warning | Findings at or above this severity get a ticket: critical \| warning. A ticket already open stays open while its finding is still there, even below this (the certificate renewal opens one for every CSR) |
| servicenow_table | incident | the table tickets are opened in (incident) |
| servicenow_assignment_group | (empty) | Values for new incidents. Names work (the API is asked to accept display values), e.g. servicenow_assignment_group: Linux Operations |
| servicenow_caller | (empty) | user name; empty = the API account |
| servicenow_category | (empty) | e.g. Software |
| servicenow_urgency | {critical: "1", warning: "2"} | a new ticket's urgency, per finding severity (your instance's values) |

| Setting | Default | What it does |
|---|---|---|
| servicenow_impact | {critical: "2", warning: "3"} | a new ticket's impact, per finding severity |
| servicenow_set_ci | true | set Configuration item = the host name (must exist in the CMDB) |
| servicenow_fields | {} | More fields for every new ticket - for an instance that makes fields such as Location, Subcategory or Channel mandatory: {column name: value}. Names and choice labels work, e.g. servicenow_fields: {location: "Building 100", subcategory: "Operating System", contact_type: "Self-service"} A field's column name: right-cli... (all of it: the role file) |
| servicenow_note_cleared | true | When a problem that had a ticket is gone, add a work note (and, if you want, resolve it). |
| servicenow_resolve_cleared | false | true = also resolve the ticket (servicenow_resolved_state, servicenow_close_code) |
| servicenow_resolved_state | "6" | Resolved |
| servicenow_close_code | "Solution provided" | must be a close code your instance has |
| servicenow_resolve_fields | {} | More fields set when a ticket is resolved (servicenow_resolve_cleared, and the test ticket), for an instance that makes them mandatory: {column name: value}, e.g. {resolved_by: svc_aap_sn} |
| servicenow_note_repeats | true | Add a work note to an open ticket each time its finding is seen again. |
| servicenow_correlation_prefix | site-health | Every ticket from these playbooks carries a correlation ID starting with this, so the same problem on the same host always updates the same ticket. |
| servicenow_validate_certs | true | check the instance's HTTPS certificate (see servicenow_ca_path) |
| servicenow_ca_path | (empty) | A CA bundle (PEM) for the instance's TLS certificate, when the execution environment does not trust its CA (e.g. a DoD root CA): a path on the controller, such as a file you add to the project: servicenow_ca_path: "{{ playbook_dir }}/files/ca/servicenow-ca.pem". Empty = the execution environment's normal trust store.... (all of it: the role file) |
| servicenow_timeout | 30 | seconds for one API request |

| Setting | Default | What it does |
|---|---|---|
| servicenow_fail_when_findings | true | Fail the job when it opened/updated tickets, so the workflow shows red while problems exist. |
| servicenow_check_mid | true | servicenow_health.yml: also check the MID Servers (needs read access to the ecc_agent table) |
| servicenow_slow_seconds | 5 | the API answering slower than this -> warning |
| servicenow_test_resolve | true | servicenow_test_ticket.yml: resolve the test incident at the end (false = leave it open to look at) |

## site_act

File: roles/site_act/defaults/main.yml

| Setting | Default | What it does |
|---|---|---|
| site_act_level | explain | ACT (GenAI) on top of ANY check playbook. The check roles already did the cheap, certain part: they found WHAT is wrong (site_findings) and know which read-only command shows more (each finding's hint). This role hands that to ACT (the act_triage role) to find out WHY, and to propose - or, if you allow it, make - th... (all of it: the role file) |
| site_act_allow | [] | self-heal only: regular expressions of fix commands ACT may run without a person. Each must match the WHOLE command. Keep them narrow, e.g. site_act_allow: ['systemctl restart (chronyd\|rsyslog\|crond)'] |
| site_act_diagnose_only_groups | [aap, aap_hosts, sat, idm, netapp, netapp_console_hosts] | Hosts where ACT may only diagnose - never fix by itself, whatever site_act_level says: AAP itself (a fix could stop the job that is running it) and vendor-managed appliances. The apply step (act_fix_approved.yml) skips them too: a person applies the fix there by hand. AAP (groups aap, aap_hosts) is ALWAYS in this list... (all of it: the role file) |
| site_act_provider | genai | ---- Which model ACT talks to ---------- site_act_provider genai GenAI.mil (the default) asksage Ask Sage (set site_act_url to your organization's instance) genai-beta the GenAI.mil beta proxy (preview models) site_act_models the model to use FOR EACH provider, e.... (all of it: the role file) |

| Setting | Default | What it does |
|---|---|---|
| site_act_models | {genai: gemini-3.8-flash, asksage: gpt-5.6-sol-gov} | the model for each provider (see above): the model follows the provider picked |
| site_act_urls | {} | the API URL for each provider; a provider not listed uses ACT's default |
| site_act_model | (empty) | one model for this run, whatever the provider (e.g. a survey answer); empty = site_act_models |
| site_act_url | (empty) | one URL for this run; empty = site_act_urls |
| site_act_ca | (empty) | a CA bundle file ON THE HOSTS for the provider's certificate; empty = the host's trust store |
| site_act_env | {} | Any other ACT environment settings (advanced; these win over the ones above), e.g. {HTTPS_PROXY: "http://proxy.yoursite.mil:8080"}. See the top of vendor/act/act for the list. Every ACT job streams the model's answer (ACT_STREAM=1), so a firewall or proxy idle timeout does not cut a long answer ("connection reset by p... (all of it: the role file) |
| site_act_model_timeout | 600 | Seconds the model may take for ONE answer, retries after a provider hiccup included (ACT's GENAI_TIMEOUT; ACT's own default, 90, is too short for a long analysis or a busy provider). Every ACT job uses it: the runbooks' use_act, the VMware alarms and capacity analyses, service watch. |
| site_act_timeout | 1200 | seconds for one whole ACT run (several answers): keep it above site_act_model_timeout |
| site_act_concurrency | 3 | How many hosts may run ACT at the same time in one job (0 = no limit: as many as the job's forks). Every host in a job uses the same key, and GenAI.mil's default quota is 60 requests and 200,000 tokens a minute per key: a fleet run with many hosts at once gets HTTP 429 (too many requests). |
| site_act_extra_instructions | (empty) | anything ACT should know about this site |
| site_act_evidence_text | (empty) | extra evidence (troubleshoot.yml passes what it collected) |

## site_email

File: roles/site_email/defaults/main.yml

**Note**
EMAIL A REPORT through your mail relay (SMTP). Off until report_email_to has an address. Used by every report job: health checks, certificate reports, database health, POA&M, ServiceNow health, troubleshooting, the VMware jobs (docs/EMAIL_REPORTS.md). The relay settings belong in playbooks/group_vars/all.yml. If the relay needs a login, attach an "SMTP relay" credential (aap/credential_types/smtp_relay.yml) - never put a password in these settings.

| Setting | Default | What it does |
|---|---|---|
| `report_email_to` | `[]` | Who gets it: a list, or text with commas / spaces / one per line (a survey answer). Empty = no email. Set it on each TEMPLATE (its Variables, or a survey): in all.yml every report job emails. |
| `report_email_cc` | `[]` | copied: a list, or text like report_email_to |
| `report_email_from` | (empty) | the sender, e.g. aap-noreply@yoursite.mil |
| `report_email_smtp_host` | (empty) | your mail relay, e.g. smtp.yoursite.mil |
| `report_email_smtp_port` | `25` | 25 or 587 with starttls, 465 with ssl |
| `report_email_security` | `starttls` | starttls = encrypt after connecting (the relay must offer it); ssl = encrypted from the start (port 465); none = not encrypted (only for a relay without encryption; then no login is sent). |
| `report_email_ca_path` | (empty) | a CA file that signed the relay's certificate (empty = the CAs the execution environment trusts) |
| `report_email_subject_prefix` | `"[AAP]"` | put before every subject, e.g. "[AAP] Certificate report: ..." |
| `report_email_html` | `true` | Send a formatted (HTML) version too, for reports that come with one (all VMware jobs). The email always carries a plain-text copy for mail clients that do not show HTML. false = plain text only. |
| `report_email_html_template` | `report.html.j2` | The look of the HTML version. Your own: copy roles/site_email/templates/report.html.j2 (e.g. to playbooks/files/email/report.html.j2), change it, and set report_email_html_template: "{{ playbook_dir }}/files/email/report.html.j2" |

## site_findings

File: `roles/site_findings/defaults/main.yml`

**Note**
Shared by every check playbook. A "finding" is one thing that is wrong on one host: check: the check that found it (disk, services, mariadb, ...) id: a stable name for this problem on this host, e.g. disk:/var. ServiceNow uses it to update the same ticket instead of opening a new one every run. severity: critical \| warning summary: one line a person understands ("/var is 96% full (critical at 95%)") hint: a read-only command that shows more. A person can paste it into a shell; ACT (when you add it later) runs it to collect evidence. Check roles only ADD findings. This role starts the list, reports it, and decides pass/fail.

| Setting | Default | What it does |
|---|---|---|
| site_fail_on | [critical, warning] | Which severities make the job FAIL. A failed job is what sends an AAP workflow down its "on failure" path (open tickets, ask ACT, wait for approval). [] = report only, never fail. |
| site_unreachable_fails | false | A host that cannot be reached (switched off, no network, ssh or WinRM refused, its login refused) in the health checks, the certificate report and renewal, database health, the service watch and troubleshoot: listed as NOT REACHED and left out, and the job is not failed for it. It still counts as checked-for, so the S... (all of it: the role file) |
| site_syslog | true | One syslog line per finding (tag below), so your log server / SIEM sees them too. |
| site_syslog_tag | site-health | the syslog tag those lines carry (to filter on in the SIEM) |
| site_eventlog_source | site-health | Windows: one Application event log entry per finding instead of syslog (source below). |
| site_os | linux | ---- Linux or Windows (set by the playbook; do not set) ---------- Linux playbooks run the roles check_<name>, the Windows ones (win_*.yml) the roles win_check_<name>. |
| site_check_role_prefix | "{{ 'win_check_' if site_os == 'windows' else 'check_' }}" | check_ (Linux) or win_check_ (Windows): set by the playbook from site_os |
| site_windows_connections | [winrm, psrp, ansible.builtin.winrm, ansible.builtin.psrp] | How a host is recognized as Windows: its connection, or its shell (Windows OpenSSH). |
| site_settings_notice | true | A notice in the job output when a project settings file (playbooks/group_vars/*.yml) defines a setting: it wins over AAP's inventory Variables box. false = no notice. |

## stigman_stack

File: roles/stigman_stack/defaults/main.yml

**Note**
STIG Manager stack on podman, managed by systemd through Quadlet unit files: stigman-mysql (MySQL 8.4) <- stigman-api (STIG Manager) <- stigman-nginx (TLS, port 443) Keycloak is EXTERNAL: the realm already exists and this role never touches it. Needs podman >= 4.4 (Quadlet): RHEL 9.2+ / RHEL 8.8+.

| Setting | Default | What it does |
|---|---|---|
| `stigman_scope` | `system` | system = rootful (/etc/containers/systemd, systemctl) <- production user = rootless (~/.config/containers/systemd, systemctl --user) <- tests / rootless hosts |
| `stigman_images` | `mysql: docker.io/library/mysql:8.4 api: docker.io/nuwcdivnpt/stig-manager:1.6.17 # pin a…` | the container images (pin STIG Manager's version; change it deliberately) |
| `stigman_config_dir` | `/etc/stigman` | nginx.conf, TLS files, CA bundle |
| `stigman_https_port` | `443` | host port nginx publishes |
| `stigman_server_name` | `"{{ inventory_hostname }}"` | the name nginx answers to (and the TLS certificate is for) |
| `stigman_oidc_provider` | `"https://keycloak.example.mil/realms/stigman"` | Existing Keycloak realm (server-side URL used by the API; browser-side URL for the web client) |
| `stigman_client_oidc_provider` | `"{{ stigman_oidc_provider }}"` | the Keycloak realm URL the browser uses (when it differs from the server-side one) |
| `stigman_client_id` | `stig-manager` | the Keycloak client the web app logs in with |
| `stigman_classification` | `U` | banner: U, CUI, C, S, TS, SCI, NONE |
| `stigman_db_schema` | `stigman` | the MySQL database STIG Manager uses |
| `stigman_db_user` | `stigman` | the MySQL account STIG Manager uses (its password: the credential) |
| `stigman_mysql_buffer_pool` | `1G` | STIG Manager docs: >= 8G for real deployments |
| `stigman_ready_retries` | `40` | How long to wait for the API to answer through nginx after (re)starting |
| `stigman_ready_delay` | `6` | seconds between those tries |

| Setting | Default | What it does |
|---|---|---|
| stigman_mysql_root_password | "{{ lookup('ansible.builtin.env', 'STIGMAN_MYSQL_ROOT_PASSWORD') }}" | ---- Secrets: NEVER set these in a vars file or inventory ---------- They come from AAP credentials (aap/credential_types/*.yml) as environment variables / files on the controller (execution environment). See docs/SECRETS.md. |
| stigman_db_password | "{{ lookup('ansible.builtin.env', 'STIGMAN_DB_PASSWORD') }}" | from the credential (STIGMAN_DB_PASSWORD), never set here |
| stigman_tls_cert_src | "{{ lookup('ansible.builtin.env', 'TLS_CERT_FILE') }}" | nginx's certificate file, from the TLS certificate credential (TLS_CERT_FILE) |
| stigman_tls_key_src | "{{ lookup('ansible.builtin.env', 'TLS_KEY_FILE') }}" | its private key file, from the credential (TLS_KEY_FILE) |
| stigman_tls_ca_src | "{{ lookup('ansible.builtin.env', 'TLS_CA_FILE') }}" | the CA bundle file, from the credential (TLS_CA_FILE) |

## troubleshoot

File: roles/troubleshoot/defaults/main.yml

| Setting | Default | What it does |
|---|---|---|
| ts_area | overview | TROUBLESHOOT: the commands an experienced admin runs first for a kind of problem, run for you (read-only), each with a line saying what it shows. The matching health checks run too, so you also get plain-words findings. Pick the area; for "service" and "network" say what. ts_area: overview \| disk \| performance \| servi... (all of it: the role file) |
| ts_service | (empty) | the systemd unit, for ts_area service (see above) |
| ts_target | (empty) | host:port, for ts_area network |
| ts_since | "-2h" | how far back logs are read (journalctl --since) |
| ts_output_lines | 60 | keep the last this-many lines of each command |
| ts_areas | overview: checks: [disk, mounts, services, performance, time, logging] commands: - {why:… | What each area runs. why is printed above the command's output. Commands must be read-only. |

## vmware_vm

File: `roles/vmware_vm/defaults/main.yml`

> **Note**
> VMWARE VMs: jobs that run against vCenter (not on the VMs), with the certified vmware.vmware collection. vCenter address and account come from a "VMware vCenter" credential on the job template (VMWARE_HOST, VMWARE_USER, VMWARE_PASSWORD). docs/VMWARE.md

| Setting | Default | What it does |
|---|---|---|
| `vm_names` | `[]` | The VMs, by their exact name in vCenter: a list, or (from a survey) one per line. No wildcards. |
| `vmware_datacenter` | (empty) | only needed when the same VM name exists in two datacenters |
| `vmware_validate_certs` | `true` | Check vCenter's certificate. false only if the execution environment does not trust your CA yet. |
| `vmware_vcenters` | `[]` | Several vCenters (e.g. Enhanced Linked Mode), one report: the reports and the ESXi security settings read the credential's vCenter AND these (names, or name:port), with the same account. The same vCenter under two names is read once. docs/VMWARE.md, "Several vCenters". e.g. [vc01.yoursite.mil, vc02.yoursite.mil] |
| `vmware_vcenter_fail_unread` | `false` | true = the job shows failed when a vCenter could not be read (the report lists it in red either way) |
| `vmware_protected_groups` | `[aap, aap_hosts]` | The AAP server is ALWAYS refused (hosts in the groups aap / aap_hosts, matched by VM name, guest host name and IP address), and so are the hosts of these groups and these VM names (* and ? allowed: DC*). List the AAP VM's name here when it differs from its host name, and vCenter's own VM. The snapshot cleanup never de... (all of it: the role file) |
| `vmware_protected_vms` | `[]` | e.g. [AAP01, VCSA01, DC*] (* and ? allowed) |
| `vm_power_mode` | `guest` | guest = ask the operating system (needs VMware Tools running; services stop cleanly) hard = reset / power off, like the button on a physical server (unsaved data is lost) |

| Setting | Default | What it does |
|---|---|---|
| vm_power_timeout | 600 | shut down: seconds to wait for the VM to be off |
| vm_snapshot_name | (empty) | The snapshot's name. Taking one: empty = aap-<date>-<time>. Deleting: the exact name to delete. |
| vm_snapshot_description | (empty) | the AAP job number and user are added |
| vm_snapshot_quiesce | false | true = VMware Tools freezes the file systems first |
| vm_snapshot_delete_all | false | delete: true = every snapshot of the VM (vm_snapshot_name empty) |
| vm_notes | (empty) | The text to write. |
| vm_notes_mode | append | append (a new line, with date and user) \| replace \| clear |
| vm_nic | 1 | Which adapter: 1 = Network adapter 1, 2, ... |
| vm_portgroup | (empty) | the port group (standard or distributed) to move it to |
| vm_secure_boot_folder | (empty) | Only the VMs in this vCenter folder ("" = every VM, or the ones in vm_names). |
| vm_secure_boot_fail | false | The job stays green (successful) when it finds VMs to fix: the report and email say what. true = mark the job FAILED when a VM is BIOS or has secure boot off, so a workflow's "on failure" path can react (open a ticket). |
| vm_datastore_warn_pct | 85 | Used space (% of capacity): a datastore at or above these is listed as a warning / critical. |
| vm_datastore_crit_pct | 90 | a datastore this % used or more is red (critical); amber from vm_datastore_warn_pct |
| vm_datastore_fail | false | The job stays green when datastores are full: the report and email say which. true = mark the job FAILED when one is at or above vm_datastore_warn_pct (or inaccessible), for workflows. |
| vm_snapshot_max_age_days | 3 | Snapshots at least this many days old: listed apart by the report, deleted by the cleanup. |

| Setting | Default | What it does |
|---|---|---|
| vm_snapshot_keep_regex | '(?i)keep\|do.?not.?delete' | Never deleted by the cleanup (the report lists them, with the reason): - a snapshot whose name or description matches this |
| vm_snapshot_cleanup_exclude_vms | [] | - a snapshot of these VMs: names, * and ? allowed (e.g. [DC*, VCSA01, AAP01]). The VMs in vmware_protected_vms and the AAP server are never cleaned up either. |
| vm_snapshot_cleanup_max_size_gb | 1024 | - a snapshot bigger than this many GB (1024 = 1 TB), or whose size cannot be read: merging a very big snapshot can take hours and slow the VM - do those by hand, at a quiet time. |
| vm_snapshot_cleanup_confirm | false | The cleanup on its own (not after a workflow approval) deletes only when this is true - e.g. a survey question "Delete them?". false = it lists what it would delete, and deletes nothing. |
| vm_snapshot_cleanup_max | 50 | Refuse to delete more than this many snapshots in one run (a typo in the age would otherwise delete them all). Raise it on purpose for a big cleanup. |
| vm_snapshot_delete_timeout | 1800 | seconds to wait for one deletion (vCenter merges it into the disk) |
| vm_snapshot_cleanup_email_only_if_deleted | false | The cleanup's email (report_email_to): true = only when it deleted something (or a deletion failed). |
| vm_alarm_types | [vcenter, datacenter, cluster, host] | Whose triggered alarms and configuration issues: vcenter, datacenter, cluster, host. Add vm, datastore or network for theirs too (VM alarms can be many). |
| vm_alarm_hours | 24 | The events of the last this-many hours: every error and warning, failed logins, host connection events and VM shutdowns / power-offs / resets / HA restarts. 0 = alarms and host state only. |
| vm_alarm_max_events | 5000 | read at most this many events (the newest) |

| Setting | Default | What it does |
|---|---|---|
| vm_alarm_max_alarm_changes | 2000 | read at most this many alarm status changes (the newest; 0 = no "Alarms that fired") |
| vm_alarm_max_rows | 200 | rows per table in the report (the artifacts have them all) |
| vm_alarm_login_critical | 10 | This many failed logins of one user from one place, or more, is critical (fewer: a warning). |
| vm_alarm_login_event_types | [BadUsernameSessionEvent, com.vmware.sso.LoginFailure, esx.audit.account.locked, esx.audi… | The event types read whatever their category (vCenter's event type names). Add to these lists rather than replacing them; [] = none of that kind. |
| vm_alarm_connection_event_types | [HostConnectionLostEvent, HostNotRespondingEvent, HostDisconnectedEvent, HostReconnection… | Events about a host losing (or failing to get back) its connection to vCenter: the Host connection table. |
| vm_alarm_vm_event_types | [VmGuestShutdownEvent, VmGuestRebootEvent, VmPoweredOffEvent, VmResettingEvent, VmFailove… | Events about a VM stopping, restarting or failing over (guest shutdown, power off, HA restart): the VM events table. |
| vm_alarm_fail | false | The job stays green when it finds alarms: the report and email say what. true = mark the job FAILED when there is a critical alarm or a host is not connected, for workflows. |
| vm_alarm_act_max_items | 40 | ACT analysis (playbooks/vm_alarm_act_analysis.yml). ACT runs on the AAP side (the execution environment), never on the ESXi hosts; the model provider, model, URL, proxy and CA are the site_act_* settings (roles/site_act/defaults/main.yml), the key an "ACT model key" credential. analyze at most this many problems, the… (all of it: the role file) |
| vm_alarm_act_config_issues | true | configuration issues too (false = alarms and events only) |
| vm_alarm_act_email_if_none | false | true = email even when there is nothing to analyze |
| vm_alarm_act_check_url | true | first check that the AAP node can reach the model's URL |

| Setting | Default | What it does |
|---|---|---|
| vm_alarm_act_model_timeout | "{{ site_act_model_timeout \| default(600) }}" | Seconds the model may take for its answer (ACT's GENAI_TIMEOUT, 90 by default, is too short for a long analysis): site_act_model_timeout (600) unless set here. A GENAI_TIMEOUT in site_act_env wins. |
| vm_health_require_ha | true | A cluster of 2 or more hosts without vSphere HA is a warning (false = HA off is on purpose). |
| vm_health_require_drs | true | A cluster of 2 or more hosts without DRS is a warning (false where DRS is off on purpose or not licensed, e.g. vSphere Standard). |
| vm_health_cpu_warn_pct | 85 | CPU and memory used, in percent of a host (or of a cluster's hosts running VMs): CPU used: a warning from this percent |
| vm_health_cpu_crit_pct | 95 | CPU used: critical from this percent |
| vm_health_mem_warn_pct | 90 | memory used: a warning from this percent |
| vm_health_mem_crit_pct | 95 | memory used: critical from this percent |
| vm_health_sensors | true | read the hosts' hardware sensors (false = skip them) |
| vm_health_max_rows | 500 | rows per table in the report (the artifacts have them all) |
| vm_health_fail | false | The job stays green when it finds problems: the report and email say what. true = mark the job FAILED when there is a critical problem (a host not connected, an HA agent isolated ...), for workflows. |
| vm_created_days | 7 | The window: the VMs created in the last this many days, up to the moment the job runs (7 = the past week; run weekly, each VM is in exactly one report). VMs created and deleted since are listed too, as long as vCenter keeps their events (often 30 days). |
| esxi_security_hosts | [] | Which hosts: names (* and ? allowed; the short name matches too). [] = every host of vCenter (or of vmware_datacenter). |

| Setting | Default | What it does |
|---|---|---|
| esxi_security_clusters | [] | only the hosts of these clusters (names, wildcards); [] = all |
| esxi_security_exclude_hosts | [] | Never these hosts (names, wildcards). E.g. a host you opened SSH on for a support case: the nightly run would otherwise close it again. |
| esxi_security_ssh | disabled | The SSH service (TSM-SSH): disabled = stopped, and set to start manually; enabled = started, and set to start with the host; "" = leave it as it is. |
| esxi_security_shell | disabled | The ESXi Shell service (TSM), the host's local shell: the same choices. |
| esxi_security_settings | UserVars.ESXiShellTimeOut: 600 UserVars.ESXiShellInteractiveTimeOut: 600 | Advanced settings, name: value; {} = none. The defaults (seconds): UserVars.ESXiShellTimeOut SSH and the ESXi Shell stop by themselves this long after someone starts them (0 = never) UserVars.ESXiShellInteractiveTimeOut an idle SSH or ESXi Shell session is logged out Others often set: UserVars.DcuiTimeOut (an idle DCU... (all of it: the role file) |
| esxi_security_lockdown | normal | Lockdown mode: normal = the host is managed only through vCenter (the DCUI still works); disabled; "" = leave it as it is. strict is refused, and a host already strict is left alone. |
| esxi_security_lockdown_exception_users | [] | Accounts that must keep logging in to the hosts directly under lockdown (a vulnerability scanner, a monitoring tool): host-local accounts, or Active Directory accounts with their own permission on the host (not vsphere.local, not an AD group). Added before lockdown is turned on; never removed. |
| esxi_security_email_only_if_changed | true | The email (report_email_to): true = only when a host was changed or failed; false = every run. |
| vm_capacity_clusters | [] | ---- vSphere capacity planning (playbooks/vm_capacity_report.yml) ---- only these clusters (names; * and ? allowed); [] = every cluster |

| Setting | Default | What it does |
|---|---|---|
| vm_capacity_history_days | 90 | The history: vCenter's own daily statistics (cluster CPU and memory, datastore space), kept a year at statistics level 1. Read this many days; the growth is a straight line over the last vm_capacity_trend_days, with at least vm_capacity_min_samples daily values. |
| vm_capacity_trend_days | 30 | the growth (straight line) is fitted on this many most recent days |
| vm_capacity_min_samples | 7 | fewer daily values than this in those days = 'not enough history' (no trend) |
| vm_capacity_events_days | 30 | VMs added and removed (vCenter keeps events 30 days by default) |
| vm_capacity_failover_hosts | 1 | Plan for this many failed hosts per cluster (N+1); a larger HA admission control failover level wins. |
| vm_capacity_mem_target_pct | 80 | memory used at most this %, with those hosts failed |
| vm_capacity_cpu_target_pct | 80 | CPU used at most this %, with those hosts failed |
| vm_capacity_vcpu_per_core_max | 4 | vCPUs per physical core (amber from here, red from 1.5 times it) |
| vm_capacity_horizon_months | 12 | Hosts recommended per cluster (of its average host size) to keep memory and CPU at their targets with those hosts failed: now, and at the current growth within this many months. |
| vm_capacity_storage_warn_pct | 85 | datastore used %: amber |
| vm_capacity_storage_crit_pct | 90 | red, and the target of the storage runway |
| vm_capacity_runway_warn_days | 90 | a runway shorter than this is amber |
| vm_capacity_runway_crit_days | 30 | and red |
| vm_capacity_include_local_datastores | false | a host's own (not shared) datastores count too |
| vm_capacity_max_rows | 100 | rows per table (problems first); the artifacts have all |

| Setting | Default | What it does |
|---|---|---|
| vm_capacity_fail | false | The job stays green: the report says what is short. true = mark it FAILED when a cluster or datastore is critical (for workflows). |
| vm_capacity_act | false | GenAI: ACT reads the numbers and makes its own estimate, shown next to the job's (math vs GenAI). Needs an "ACT model key" credential on the template; the same site_act_* settings as the alarms analysis. ACT runs on the AAP side and runs no command. |
| vm_capacity_act_max_items | 30 | items ACT estimates: everything overall, then clusters and datastores, problems first |
| vm_capacity_act_model_timeout | "{{ site_act_model_timeout \| default(600) }}" | seconds the model may take for its answer |
| vm_capacity_act_check_url | true | first check that the AAP node can reach the model's URL |
| vm_deploy_ticket | (empty) | ---- Deploy a VM from a ServiceNow ticket (playbooks/vm_deploy_from_ticket.yml) ---- The ticket (survey answer): INC..., TASK..., SCTASK..., RITM..., CHG... - or your instance's own prefixes (INCN..., CHGN...): the job asks ServiceNow which table a ticket is in. The ticket says what to build in 'key: value' lines - in... (all of it: the role file) |
| vm_deploy_ticket_table | (empty) | leave it empty (the job finds each ticket's table); a table of your own, e.g. u_vm_request: for numbers whose prefix the job does not know (CTASK stays change_task) |
| vm_deploy_ticket_prefixes | {} | Your instance's own number prefixes and their tables, e.g. {INCN: incident, CHGN: change_request}. Rarely needed: for a prefix it does not know, the job asks ServiceNow's task table (and takes INCN as INC, CHGN as CHG when it may not read that table). A table as people say it is understood: change / change request ->... (all of it: the role file) |

| Setting | Default | What it does |
|---|---|---|
| vm_deploy_parent_tables | [change_request] | The request may also be on the ticket's parent - e.g. a task (SCTASK, CTASK) whose change holds the requirements: the parent's 'key: value' lines (and, with vm_deploy_plain_words, its plain words) are read too, under the ticket's own (what the ticket says wins). A parent that cannot be read, is closed, or is not ready... (all of it: the role file) |
| vm_deploy_parent_fields | [change_request, parent] | The ticket's fields that point to its parent - the first one set is used: a change task's change_request, any ticket's parent (or your instance's own field, e.g. u_change): |
| vm_deploy_parent_text_fields | [description] | The parent's fields with the request, in this order (a later one wins), e.g. [description, implementation_plan]; comments and work_notes may be listed too (oldest first). GenAI reads them as well. |
| vm_deploy_ready_by_table | change_request: {approval: Approved, state: Implement} | Before anything is built, a ticket of these tables - the ticket itself, or the parent it belongs to (a change task's change) - must have these fields: one value or a list (any of them), written as the ticket's form shows them (upper / lower case ignored). Checked by the dry run, the propose step AND right before the b... (all of it: the role file) |
| vm_deploy_templates | [] | What a ticket may ask for (names; * and ? allowed). Templates MUST be listed: nothing else is deployed. The others: [] = any. e.g. [rhel9-gold, win2022-gold] |
| vm_deploy_clusters | [] | clusters (and standalone hosts) a ticket may use; also the candidates of automatic placement |
| vm_deploy_datastores | [] | datastores (or datastore clusters) a ticket may use; also the candidates of automatic placement |
| vm_deploy_folders | [] | VM folders a ticket may use (paths below the datacenter's VM folder, e.g. Linux/*) |
| vm_deploy_max_cpu | 16 | the most vCPUs a ticket may ask for |

| Setting | Default | What it does |
|---|---|---|
| vm_deploy_max_memory_gb | 128 | the most memory a ticket may ask for, in GB |
| vm_deploy_max_vms | 10 | A ticket may ask for several VMs ("2 VMs named web01 and web02", "vm_name: web01, web02", or "Hostnames:" and one name per line): one VM each, the same size - or each its own, from a line of its own - built one after the other, the ticket closed when all are built. More names than this: refused, nothing built (the lim... (all of it: the role file) |
| vm_deploy_name_regex | '^[A-Za-z0-9][A-Za-z0-9-]{0,62}$' | The VM names allowed (a regular expression); e.g. '^[a-z0-9-]{1,15}$' for Windows-safe names. |
| vm_deploy_assignment_group | (empty) | Only tickets assigned to this group (its name as ServiceNow shows it), or to one of a list of groups (["Group A", "Group B"]); "" = any group (the pickup needs one). |
| vm_deploy_defaults | {} | What a ticket does not say: e.g. {cluster: PROD-CL01, datastore: DS-CLUSTER-01, folder: Linux/New}. Not set = the template's own cluster, datastore and folder; its CPU and memory. |
| vm_deploy_sites | {} | Sites: a ticket's 'site: A' line (or its Location field, or with plain words a site's name or its words in vm_deploy_plain_sites - each VM by its own line) picks the placement - over vm_deploy_defaults and the name rules; a cluster / datastore / folder the ticket names itself still wins. A site in another vCenter: its... (all of it: the role file) |
| vm_deploy_site_required | true | with sites set up: a ticket must say its site (false = vm_deploy_defaults); a name rule counts too |

| Setting | Default | What it does |
|---|---|---|
| vm_deploy_name_rules | [] | Name rules: the VM's name picks where it goes - the first rule whose 'match' fits the name (* and ?, upper / lower case alike) fills in what the ticket does not say: a cluster (and datastores, folder ...) or a site. The ticket's own lines, and a site the ticket names, still win. A name used at several sites: 'sites' -... (all of it: the role file) |
| vm_deploy_pxe_templates | [] | Templates with no operating system yet, that install it themselves: they boot from the network (PXE, e.g. an MECM task sequence) or from their CD / ISO (e.g. a kickstart). No guest customization, no waiting for an IP address; the MAC address and the computer name go on the ticket. e.g. [win2022-pxe] |
| vm_deploy_guestinfo | true | Give a VM from one of those empty templates the ticket's name and number as VM settings its installer can read: guestinfo.hostname and guestinfo.ticket (a kickstart with VMware Tools in the installer: vmtoolsd --cmd "info-get guestinfo.hostname"). Needs the vCenter privilege Change configuration > Advanced configurati... (all of it: the role file) |
| vm_deploy_template_aliases | {} | Short names for long template names - for the ticket's 'template:' line and GenAI's catalog (whose keys may be these short names): {short name: the template's name in vCenter}, e.g. {win2022: "Windows 2022 Standard Template - Site 1", rhel9: "RHEL 9 Server Template"} A ticket may also write a template's own name in ot... (all of it: the role file) |
| vm_deploy_template_specs | {} | The customization specification (vCenter > Policies and Profiles) for each template, when the ticket, its site and the name rules name none: {template name or pattern: spec}. A Windows template that is not network-boot needs one (Sysprep: administrator password, domain or workgroup), e.g. {"Windows 2022*": Win2022-Sys... (all of it: the role file) |

| Setting | Default | What it does |
|---|---|---|
| vm_deploy_max_disks | 4 | disks a ticket may ask for (disks_gb: 100, 200 = disk 1 grown to 100 GB, one 200 GB disk added) |
| vm_deploy_max_disk_gb | 2048 | the biggest disk a ticket may ask for, in GB |
| vm_deploy_max_total_disk_gb | 4096 | all its disks together, in GB |
| vm_deploy_mem_target_pct | 80 | Automatic placement - when the ticket, its site and vm_deploy_defaults name no cluster / datastore: the job chooses among a site's 'clusters' / 'datastores' lists (e.g. SiteA: {clusters: [A-PROD-*]}), else among vm_deploy_clusters / vm_deploy_datastores; with neither, the template's own. a cluster fits when its memory... (all of it: the role file) |
| vm_deploy_failover_hosts | 1 | hosts to plan as failed when choosing a cluster |
| vm_deploy_datastore_max_used_pct | 85 | a datastore fits when it stays under this % with the VM at its full size |
| vm_deploy_domain | localdomain | the DNS domain Linux customization sets (a ticket's domain: wins) |
| vm_deploy_customize | true | guest customization: host name = vm_name, every adapter on DHCP |
| vm_deploy_power_on | true | power the new VM on (false = leave it off, e.g. to check it first) |
| vm_deploy_wait_customization | 900 | seconds to wait for vCenter's customization result |
| vm_deploy_wait_ip | 600 | seconds to wait for an IPv4 address (DHCP, through VMware Tools) |
| vm_deploy_done_fields | {} | Fields set on the ticket when the VM is up, e.g. to resolve an incident: {state: "6", close_code: "Solution provided", close_notes: "VM deployed"} (your instance's values) |

| Setting | Default | What it does |
|---|---|---|
| vm_deploy_done_fields_by_table | {} | The same for one table (the ticket's table: incident, sc_task, change_task ...), in place of vm_deploy_done_fields for it - e.g. close the task, never the change it belongs to: {sc_task: {state: "3", close_notes: "VM deployed by AAP"}} (3 = Closed Complete; your instance's values) Sent on its own, after the result not... (all of it: the role file) |
| vm_deploy_done_fields_issues_by_table | {} | Over those fields when every VM is built but something did not finish: no IPv4 address in time, no customization result, the guest's host name not applied, no network at power on, guestinfo not set - e.g. {change_task: {close_code: successful_issues}} (your instance's values; {} = closed the same way). What happened i... (all of it: the role file) |
| vm_deploy_done_fields_pxe | true | Also for a VM from a network-boot template (vm_deploy_pxe_templates), which is powered on and still installing its OS from MECM when the job ends. false = that ticket stays open for whoever checks the install. |
| vm_deploy_plain_words | false | ---- A request in plain words, without GenAI (off by default) ---- true = also read the ticket's own words, by the rules below, so a requester can write "a Red Hat 9 server named ab01web05, medium" instead of 'key: value' lines. Never a guess: two different answers in the text are a question to the requester (a commen... (all of it: the role file) |
| vm_deploy_plain_templates | {} | The words for each template (any case; "windows 2022" also matches "Windows Server 2022", "server 2025" also "a new 2025 VM"). An allowed template's own name and its short names (vm_deploy_template_aliases) count too. An OS no template is for (Server 2025, only a 2022 template) is a question - never another version; t... (all of it: the role file) |

| Setting | Default | What it does |
|---|---|---|
| vm_deploy_plain_sizes | {} | Size words and what they mean, e.g. {small: {cpu: 2, memory_gb: 8}, medium: {cpu: 4, memory_gb: 16}, large: {cpu: 8, memory_gb: 32}}. "4 vCPU" and "16 GB RAM" in the ticket are read too, and win in the same text. |
| vm_deploy_plain_sites | {} | The words for each site (vm_deploy_sites), as people write the place - any case, the spaces optional ("Site Two" is also "SiteTwo"). A site's own name counts too. A change that lists its servers one per line ('new VM / ab01web05 / ... Location: Site Two') puts each VM at the site its own line names; the CPU, memory an... (all of it: the role file) |
| vm_deploy_genai | false | ---- GenAI reads a request written in plain words (a proof of concept, off by default) ---- When the ticket's own 'key: value' lines do not give vm_name and template: ACT (GenAI, evidence mode, no commands) copies out what the ticket says - the names, the operating system as written, CPU, memory, disks - each with the... (all of it: the role file) |
| vm_deploy_template_catalog | {} | Optional: templates GenAI may name by key, each with a plain description (the key: the template's exact name). Not needed when the templates' names say their OS: the job finds the template from the OS the ticket names. A key for another version than the ticket asks for is dropped. e.g. {win2022-pxe: "Windows Server 20... (all of it: the role file) |
| vm_deploy_genai_approval | true | true: a GenAI proposal waits for a person - the workflow's approval (or a second run) - before it is built. false: one run reads, checks and builds, when every value is traced to the ticket and within the rules (questions or a refusal still stop it). |

| Setting | Default | What it does |
|---|---|---|
| vm_deploy_phase | (empty) | Which step this run is: "" = one job does it all; propose = the workflow's first step (read, ask GenAI if needed, check, write where it would go on the ticket - build nothing); build = the step after the approval (build what the ticket and its proposal say - never ask GenAI). |
| vm_deploy_genai_model_timeout | "{{ site_act_model_timeout \| default(600) }}" | seconds the model may take for its answer |
| vm_deploy_poll_enabled | true | ---- Pick up VM request tickets on a schedule (playbooks/vm_deploy_poll.yml) ---- Every run (an AAP schedule, e.g. every 15 minutes) asks ServiceNow for NEW VM requests - active, assigned to vm_deploy_assignment_group (required), marked by vm_deploy_poll_prefix and/or vm_deploy_poll_query - and starts the deploy workf... (all of it: the role file) |
| vm_deploy_poll_tables | [incident] | the ServiceNow tables to look in: incident, sc_task, task ... |
| vm_deploy_poll_prefix | "VM request" | a VM request's short description starts with this ("" = mark them by vm_deploy_poll_query) |
| vm_deploy_poll_query | (empty) | An extra ServiceNow encoded query, ANDed with the rest, e.g. "category=vm_request" or "cmdb_ci.name=VM Request" ("" = none). ^NQ (a second query) is refused. |
| vm_deploy_poll_states | ["1"] | the ticket states to pick up (state values: "1" = New) |
| vm_deploy_poll_max | 3 | the most tickets started per run; the rest wait for the next run |
| vm_deploy_poll_scan | 100 | the most tickets read per table and run, oldest first (tickets AAP already has count too) |
| vm_deploy_poll_workflow | "VM - deploy from ServiceNow ticket (workflow)" | the workflow job template started per ticket |
| vm_deploy_poll_dry_run | false | true = only list what it would pick up: no note, no workflow (Check does the same) |

| Setting | Default | What it does |
|---|---|---|
| vm_deploy_poll_claim_fields | {} | Fields set on the ticket together with the "Picked up" note, e.g. {state: "2"} (In Progress) - so the ticket leaves the states picked up as well ({} = only the note). |
| vm_deploy_poll_aap_ca_path | (empty) | a CA file for AAP's own HTTPS certificate ("" = what the execution environment trusts) |

## win_act

File: roles/win_act/defaults/main.yml

| Setting | Default | What it does |
|---|---|---|
| win_act_dir | 'C:\ProgramData\act' | ACT FOR WINDOWS on the servers (vendor/act-windows/act.ps1, the PowerShell port of ACT). Installed into win_act_dir. That folder gets its own owner and permissions - Administrators and SYSTEM only, nothing inherited - because ordinary users may create folders and files in C:\ProgramData (and C:\temp): a non-admin coul... (all of it: the role file) |
| win_act_state | present | present \| absent (removes win_act_dir) |
| win_act_source | "{{ role_path }}/../../vendor/act-windows/act.ps1" | The ACT script this repository ships (vendor/act-windows, version in its VERSION file) |

## win_check_accounts

File: roles/win_check_accounts/defaults/main.yml

> **Note**
> WINDOWS ACCOUNTS (local accounts; domain controllers have none and are skipped) - the built-in Guest account is disabled -> critical if not - enabled local accounts: password never expires, password too old, no logon for a long time -> warning - members of the local Administrators group that you did not allow -> warning Names below allow wildcards (* and ?), e.g. 'YOURDOMAIN\Domain Admins', '*\svc_aap'.

| Setting | Default | What it does |
|---|---|---|
| win_check_accounts_password_max_age_days | 60 | an enabled local account's password older than this -> warning (0 = off) |
| win_check_accounts_inactive_days | 35 | an enabled local account with no logon for this many days -> warning (0 = off) |

| Setting | Default | What it does |
|---|---|---|
| win_check_accounts_password_never_expires_allowed | [] | accounts allowed "password never expires" (wildcards) |
| win_check_accounts_inactive_allowed | [] | accounts allowed to stay unused (wildcards) |
| win_check_accounts_admins_allowed | [] | Empty = list the members in the job output only. Set it to report anyone else, e.g. win_check_accounts_admins_allowed: ['*\Administrator', 'YOURDOMAIN\Domain Admins', 'YOURDOMAIN\svc_aap'] |

## win_check_audit

File: roles/win_check_audit/defaults/main.yml

> **Note**
> WINDOWS AUDIT: the audit policy records what the STIG asks for, the event logs are big enough, and the log forwarding agent runs. One finding lists every missing audit setting.

| Setting | Default | What it does |
|---|---|---|
| win_check_audit_subcategories | - {name: Credential Validation, guid: 0CCE923F-69AE-11D9-BED3-505054503030, need: Success… | Advanced audit policy subcategories (checked by GUID, so any Windows language works). need: Success \| Failure \| Success and Failure |
| win_check_audit_security_log_min_kb | 196608 | STIG |
| win_check_audit_application_log_min_kb | 32768 | the Application log's maximum size must be at least this (KB) |
| win_check_audit_system_log_min_kb | 32768 | the System log's maximum size must be at least this (KB) |
| win_check_audit_forwarder_services | [] | Log forwarding agents that must run, e.g. [SplunkForwarder] or [nxlog] |

## win_check_certs

File: roles/win_check_certs/defaults/main.yml

**Note**
WINDOWS CERTIFICATES: every certificate in the machine stores below, and which of them are in use (https bindings of IIS / WSUS / ..., the WinRM HTTPS listener AAP connects through, Remote Desktop). Expiring soon -> warning / critical. Expired: critical when in use, or when it expired less than win_check_certs_expired_report_days ago (older leftovers are only listed in the certificate report).

| Setting | Default | What it does |
|---|---|---|
| `win_check_certs_warn_days` | `30` | a certificate expiring within this many days -> warning (amber in the certificate report) |
| `win_check_certs_notice_days` | `60` | the certificate report colours these blue (an early heads-up; not a finding) |
| `win_check_certs_crit_days` | `7` | within this many days -> critical |
| `win_check_certs_expired_report_days` | `30` | an expired certificate NOT in use is a finding for this many days after it expired; older ones only in the report |
| `win_check_certs_stores` | `['Cert:\LocalMachine\My', 'Cert:\LocalMachine\WebHosting', 'Cert:\LocalMachine\Remote Des…` | the machine certificate stores read |
| `win_check_certs_ignore_subjects` | `[]` | wildcards, e.g. ['CN=localhost*'] |
| `win_check_certs_endpoints` | `[]` | TLS services whose certificate a client actually gets, e.g. win_check_certs_endpoints: - {name: IIS, host: 127.0.0.1, port: 443, servername: web01.yoursite.mil} |

## win_check_disk

File: `roles/win_check_disk/defaults/main.yml`

**Note**
WINDOWS DISK: how full each local fixed volume (C:, D:, ...) is.

| Setting | Default | What it does |
|---|---|---|
| `win_check_disk_warn_pct` | `85` | % used -> warning |
| `win_check_disk_crit_pct` | `95` | % used -> critical |
| `win_check_disk_min_free_gb` | `0` | also warn below this many GiB free (0 = off): big volumes |

| Setting | Default | What it does |
|---|---|---|
| win_check_disk_overrides | {} | Different limits for particular volumes, e.g. a database volume kept full on purpose: win_check_disk_overrides: 'D:': {warn: 92, crit: 98} |
| win_check_disk_ignore | [] | volumes not to check, e.g. ['E:'] |

## win_check_eventlog

File: roles/win_check_eventlog/defaults/main.yml

> **Note**
> WINDOWS EVENT LOG, the last win_check_eventlog_hours: - unexpected shutdown or power loss (41, 6008), a crash / blue screen (1001) -> critical - disk and file system errors (disk, Ntfs, storage drivers) -> critical - the Security log was cleared (1102) -> critical - many errors in System + Application (the finding names the top sources) -> warning - a System / Application / Security log that is full and not overwriting -> critical

| Setting | Default | What it does |
|---|---|---|
| win_check_eventlog_hours | 24 | how many hours back the event logs are read |
| win_check_eventlog_error_warn | 50 | this many errors or more in System + Application -> warning |

## win_check_network

File: roles/win_check_network/defaults/main.yml

> **Note**
> WINDOWS NETWORK, from the server itself: - a default gateway and DNS servers are configured; - the names you list resolve, the TCP services you list answer; - domain members: the computer's trust with the domain works (nltest /sc_query). A broken trust is the classic "the trust relationship between this workstation and the primary domain failed" at logon.

| Setting | Default | What it does |
|---|---|---|
| win_check_network_dns_names | [] | Names every Windows server must resolve, e.g. [dc01.yoursite.mil, ldap.yoursite.mil] |
| win_check_network_tcp | [] | Services every Windows server must reach, e.g. win_check_network_tcp: - {name: LDAPS, host: dc01.yoursite.mil, port: 636} - {name: SIEM, host: siem.yoursite.mil, port: 9997} |
| win_check_network_secure_channel | true | domain members: test the computer's trust with the domain (nltest /sc_query) |

| Setting | Default | What it does |
|---|---|---|
| win_check_network_timeout_ms | 3000 | milliseconds for each TCP test |

## win_check_patching

File: roles/win_check_patching/defaults/main.yml

> **Note**
> WINDOWS PATCHING (read-only; installing updates is a later runbook) - a restart is pending for installed updates (Windows Update, component servicing, ConfigMgr) - days since the last update was installed (Get-HotFix) - optional: ask Windows Update which updates are missing (slower; needs full PowerShell)

| Setting | Default | What it does |
|---|---|---|
| win_check_patching_warn_days | 35 | last update installed this many days ago or more -> warning |
| win_check_patching_crit_days | 60 | this many days or more -> critical |
| win_check_patching_reboot_pending | true | report a restart pending for installed updates |
| win_check_patching_search_pending | false | also ask Windows Update which updates are missing (slower) |

## win_check_performance

File: roles/win_check_performance/defaults/main.yml

> **Note**
> WINDOWS PERFORMANCE: CPU (average of three samples a second apart), memory, page file. The finding names the busiest processes, so you know where to look.

| Setting | Default | What it does |
|---|---|---|
| win_check_performance_cpu_warn_pct | 90 | CPU busy % (average of three samples) -> warning |
| win_check_performance_cpu_crit_pct | 98 | CPU busy % -> critical |
| win_check_performance_mem_warn_pct | 90 | memory in use % -> warning |
| win_check_performance_mem_crit_pct | 97 | memory in use % -> critical |
| win_check_performance_pagefile_warn_pct | 80 | page file in use % -> warning |

## win_check_security

File: roles/win_check_security/defaults/main.yml

> **Note**
> WINDOWS SECURITY BASELINE (a few high-value STIG items; not a full STIG scan: that is your SCAP / vulnerability scanner)

| Setting | Default | What it does |
|---|---|---|
| `win_check_security_av` | `auto` | Antivirus: auto = check Microsoft Defender when it is the active antivirus (skipped when it is passive or off because another product protects the server); defender = always; none = never. |
| `win_check_security_av_signature_max_days` | `7` | Defender's signatures older than this many days -> finding |
| `win_check_security_av_services` | `[]` | Services of your endpoint protection that must run, e.g. Trellix / McAfee: [masvc, mfemms] |
| `win_check_security_firewall_profiles` | `[Domain, Private, Public]` | must be on |
| `win_check_security_smb1` | `true` | SMBv1 must be off |
| `win_check_security_rdp_nla` | `true` | Remote Desktop must require Network Level Authentication |
| `win_check_security_uac` | `true` | User Account Control must be on |
| `win_check_security_legacy_tls` | `true` | TLS 1.0 / 1.1 must be off for incoming connections |

## win_check_services

File: `roles/win_check_services/defaults/main.yml`

> **Note**
> WINDOWS SERVICES - every required service must be running (critical); - an automatic service that stopped WITH AN ERROR (exit code not 0) is a warning. Automatic services that stop cleanly by design (trigger-start, delayed) are not reported; - a service that crashed several times recently (System log events 7031 / 7034).

| Setting | Default | What it does |
|---|---|---|
| `win_check_services_required` | `[EventLog, RpcSs, Dnscache, W32Time, WinRM, mpssvc]` | Must be running on every Windows server. Add your own per group with ..._required_extra, e.g. for IIS servers: win_check_services_required_extra: [W3SVC, WAS] |
| `win_check_services_required_extra` | `[]` | more services that must run, e.g. for a group of IIS servers (see above) |

| Setting | Default | What it does |
|---|---|---|
| win_check_services_ignore | - gupdate - gupdatem - edgeupdate - edgeupdatem - MicrosoftEdgeElevationService - GoogleU… | Automatic services never to report (wildcards allowed: * and ?) |
| win_check_services_crash_hours | 24 | look back this far for unexpected stops |
| win_check_services_crash_warn | 3 | this many unexpected stops of one service -> warning |

## win_check_time

File: roles/win_check_time/defaults/main.yml

> **Note**
> WINDOWS TIME: the Windows Time service runs, syncs with a real source (not the local clock), synced recently, and how far off the clock is from its source. Kerberos (domain logons) fails when a clock is 5 minutes off.

| Setting | Default | What it does |
|---|---|---|
| win_check_time_offset_warn_sec | 1 | seconds off its time source -> warning |
| win_check_time_offset_crit_sec | 5 | seconds off -> critical |
| win_check_time_max_hours_since_sync | 24 | last successful sync longer ago than this many hours -> finding |

## win_patch

File: roles/win_patch/defaults/main.yml

> **Note**
> WINDOWS PATCH through the ConfigMgr client: install what Software Center has for the server (the software updates deployed to it), restart if the updates need it, and prove the server came back: every automatic service that was running before runs again. One server at a time by default; the rollout stops at the first server that fails. Check mode lists and changes nothing.

| Setting | Default | What it does |
|---|---|---|
| automatic_restarts | false | NO AUTOMATIC RESTARTS unless you say so: this playbook restarts a server only when automatic_restarts is true (the job template's Variables, or the survey) and the updates need it. With automatic_restarts false (the default) it installs the updates and tells you which servers still need a restart; a person restarts th… (all of it: the role file) |

| Setting | Default | What it does |
|---|---|---|
| win_patch_never_reboot_groups | "{{ patch_never_reboot_groups \| default(['aap', 'aap_hosts', 'sat', 'netapp', 'netapp_con… | Servers in these groups are patched but NEVER restarted by this playbook (restart them by hand). Follows the Linux setting (patch_never_reboot_groups) when you set it, so ONE list covers both; the fallback is the Linux role's default (roles/patch/defaults/main.yml; tests/test_protection.py checks it). |
| win_patch_never_patch_groups | "{{ patch_never_patch_groups \| default(['no_patch', 'aap', 'aap_hosts', 'sat', 'netapp',… | Servers in these groups are skipped entirely (e.g. servers patched only by hand). |
| win_patch_scan_first | true | ask the client to rescan and re-evaluate before installing |
| win_patch_scan_wait_sec | 120 | let that scan settle |
| win_patch_timeout_min | 120 | per round: download + install |
| win_patch_poll_sec | 30 | how often to look at the progress |
| win_patch_max_rounds | 2 | After a restart, look again: servicing-stack and cumulative updates often reveal more updates. |
| win_patch_restart_delay_sec | 30 | shutdown.exe /t |
| win_patch_restart_timeout_min | 30 | how long the server may take to come back |
| win_patch_restart_settle_sec | 30 | after the port answers again, before trying to log on |
| win_patch_reconnect_pause_sec | 20 | between logon attempts after the restart |
| win_patch_services_grace_min | 5 | After the restart, give delayed-start services this long before calling one missing. |
| win_patch_services_ignore | ['gupdate*', 'edgeupdate*', 'MicrosoftEdgeElevationService', 'GoogleUpdater*', 'MapsBroke… | Automatic services that come and go on their own (wildcards); not compared before/after. |
| win_patch_winrm_host | "{{ ansible_host \| default(inventory_hostname) }}" | Where AAP reaches WinRM, to see the server go down and come back during the restart |
| win_patch_winrm_port | "{{ ansible_port \| default(5986) }}" | its WinRM port (5986 = HTTPS) |

## win_troubleshoot

File: roles/win_troubleshoot/defaults/main.yml

| Setting | Default | What it does |
|---|---|---|
| ts_area | overview | WINDOWS TROUBLESHOOT: the PowerShell commands an experienced Windows admin runs first for a kind of problem, run for you (read-only), each with a line saying what it shows. The matching health checks run too, so you also get plain-words findings. ts_area: overview \| disk \| performance \| service \| network \| login \| tim... (all of it: the role file) |
| ts_service | (empty) | the service name, for ts_area service (see above) |
| ts_target | (empty) | host:port, for ts_area network |
| ts_hours | 2 | how many hours back the event logs are read |
| ts_output_lines | 60 | keep the last this-many lines of each command |
| win_ts_areas | overview: checks: [disk, services, performance, time, eventlog] commands: - {why: "Window… | What each area runs. why is printed above the command's output. Commands must be read-only. |